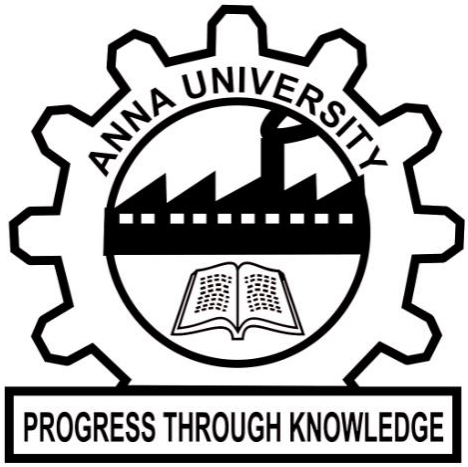
# EMPLOYEE MANAGEMENT ANALYSIS USING JAVA FULL STACK

**A SUMMER INTERNSHIP REPORT**

***Submitted by***

**SURJITH S - 810023205031**




**DEPARTMENT OF INFORMATION TECHNOLOGY**

**UNIVERSITY COLLEGE OF ENGINEERING, BIT CAMPUS, ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024**

# UNIVERSITY COLLEGE OF ENGINEERING, BIT CAMPUS, ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024

## DEPARTMENT OF INFORMATION TECHNOLOGY

## BONAFIDE CERTIFICATE

This is to certify that Summer internship project report **EMPLOYEE MANAGEMENT ANALYSIS USING JAVA FULL STACK** is the bonafide work of **SURJITH S (810023205031)** who has satisfactorily completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.Tech INFORMATION TECHNOLOGY** during the academic year 2026-2027.

**FACULTY INCHARGE** **HEAD OF THE DEPARTMENT**

# TABLE OF CONTENT


| CHAPTER NO. | TITLE |
|---|---|
| | **LIST OF FIGURES** |
| | **ABSTRACT** |
| **CHAPTER 1** | **INTRODUCTION & OBJECTIVES** |
| 1.1 | Introduction |
| 1.2 | Objectives of the Internship |
| 1.3 | Internship Particulars |
| 1.4 | Organisation Profile |
| 1.5 | Technologies Studied |
| 1.6 | Scope of the Internship |
| 1.7 | Internship Schedule |
| **CHAPTER 2** | **SYSTEM STUDY** |
| 2.1 | Problem Statement Identification |
| 2.2 | Proposed Solution |
| 2.3 | Existing System and Limitations |
| 2.4 | Functional Requirements |
| 2.5 | Non-Functional Requirements |
| 2.6 | Feasibility Study |
| 2.7 | Hardware and Software Requirements |
| 2.8 | Advantages of the Proposed System |
| **CHAPTER 3** | **ARCHITECTURE** |
| 3.1 | Architectural Design |
| 3.2 | Modular Breakdown |
| 3.3 | Database Design |
| 3.4 | Data Dictionary |
| 3.5 | Data Flow |
| 3.6 | User Roles and Access |
| 3.7 | Design Principles Followed |
| 3.8 | Page Navigation Flow |
| **CHAPTER 4** | **DEVELOPMENT AND RESULT ANALYSIS** |
| 4.1 | System Implementation and Development |
| 4.2 | Result Analysis and Testing Parameters |
| 4.3 | Test Cases |
| 4.4 | Challenges and Solutions |
| 4.5 | Output Screens |
| 4.6 | Database Output |
| 4.7 | Summary of Output Screens |
| **CHAPTER 5** | **CONCLUSION** |

| CHAPTER NO. | TITLE |
|---|---|
| 5.1 | Conclusion |
| 5.2 | Skills Gained |
| 5.3 | Future Enhancements |
| | **REFERENCES** |
| | **APPENDICES** |

# LIST OF FIGURES


| Figure No. | Title |
|---|---|
| 1.1 | Internship Schedule (June 2026) |
| 3.1 | Three-Layer Architecture of the System |
| 3.2 | Entity-Relationship Diagram |
| 3.3 | Data Flow Diagram – Level 0 (Context Diagram) |
| 3.4 | Data Flow Diagram – Level 1 |
| 3.5 | Use-Case Diagram for Admin and Viewer |
| 3.6 | Model-View-Controller Flow |
| 3.7 | Page Navigation Flow of the Application |
| 4.1 | Project Structure of the Application |
| 4.2 | Login Page |
| 4.3 | Error Message for Invalid Login |
| 4.4 | Dashboard of the Application |
| 4.5 | Employee List Page |
| 4.6 | Add Employee Form |
| 4.7 | Client-Side Validation Messages |
| 4.8 | Duplicate Employee ID Rejected |
| 4.9 | Success Message after Adding an Employee |
| 4.10 | Search Result for the Development Department |
| 4.11 | No Records Found Message |
| 4.12 | Edit Employee Form |
| 4.13 | Success Message after Updating an Employee |
| 4.14 | Confirmation Dialog before Deleting a Record |
| 4.15 | Department-wise Employee Report |
| 4.16 | Employee List for a Viewer |
| 4.17 | Access Denied Message for a Restricted Operation |
| 4.18 | Output of CREATE TABLE and INSERT Statements |
| 4.19 | Output of SELECT with WHERE and UPDATE |
| 4.20 | Output of GROUP BY with COUNT |
| 4.21 | Output of INNER JOIN between employee and department |

# ABSTRACT

The internship was successfully completed at **Besant Technologies, Chennai**, from **01 June 2026 to 30 June 2026** under the domain of **Java Full Stack Development**, with training hours from 10:00 AM to 04:00 PM. The internship provided practical exposure to web application development and helped bridge the gap between academic learning and industry practices.

During the training, knowledge was gained in **Java, HTML, CSS, JavaScript and SQL**. These technologies were used to design user interfaces, develop application logic, manage data, and perform CRUD operations. As part of the internship, a project titled **"Employee Management Analysis"** was undertaken to manage employee information through a web-based application. The internship enhanced programming, database management, analytical, and problem-solving skills, providing a strong foundation for future academic projects and professional careers in **Full Stack Java Development**.

# CHAPTER 1: INTRODUCTION & OBJECTIVES

## 1.1 Introduction

Full Stack Java Development involves developing complete web applications by integrating the front-end, back-end, and database components. It enables developers to build efficient, secure, and user-friendly applications for various real-world requirements. The front-end is developed using **HTML**, **CSS**, and **JavaScript**, where HTML provides the structure of web pages, CSS enhances their appearance, and JavaScript adds interactivity and dynamic features. The back-end is developed using **Java**, which receives requests, applies business rules, and communicates with the database.

An essential part of web application development is data management. **Structured Query Language (SQL)** is used with a **relational database** to create tables, insert, update, delete, and retrieve data efficiently, while keys and constraints help maintain data integrity and consistency.

During the internship at **Besant Technologies**, these technologies were applied to the project **Employee Management Analysis**, which focuses on efficient management of employee records through CRUD operations. The internship provided practical experience in web application development, database handling, and software testing, strengthening the foundation for future software development projects.

## 1.2 Objectives of the Internship

The primary objective of the internship was to gain practical knowledge in **Full Stack Java Development** and understand the process of developing web applications. It provided an opportunity to apply theoretical concepts in a professional training environment.

The internship focused on learning **Java, HTML, CSS, JavaScript, and SQL** for designing web pages, writing application logic, managing data, and performing CRUD operations. It also helped in understanding request-response communication, input

validation, and improving technical and analytical skills.

The internship enhanced problem-solving abilities, communication, teamwork, time management, and professional ethics. Overall, it bridged the gap between academic learning and industry practices, providing a strong foundation for future academic projects and careers in software and web application development.

## 1.3 Internship Particulars

The internship was carried out as part of the Summer Internship course (IT3711) prescribed for the seventh semester of the B.Tech Information Technology programme. The particulars of the internship, as recorded in the offer letter, internship letter and certificate of attendance issued by the organisation, are given in the table below.

| **Particular** | **Details** |
|---|---|
| Name of the Student | SURJITH S |
| Register Number | 810023205031 |
| Programme | B.Tech Information Technology |
| Institution | University College of Engineering, BIT Campus, Anna University, Tiruchirappalli – 620 024 |
| Organisation | Besant Technologies, Chennai |
| Domain | Java Full Stack Development |
| Project Title | Employee Management Analysis |
| Duration | 01 June 2026 to 30 June 2026 (1 Month) |
| Training Time | 10:00 AM to 04:00 PM |
| Role | Project Trainee |

## 1.4 Organisation Profile

**Besant Technologies** is a technology training organisation located at No. 119, 11th Main Road, Vijaya Nagar, Velachery, Chennai, Tamil Nadu – 600042. The organisation provides instructor-led training and internship programmes in software development and related technologies. Students of engineering colleges join these programmes to gain hands-on experience in tools and technologies that are used in the software industry.

For this internship, the organisation issued an **Offer Letter** (dated 10 May 2026)

confirming the Java Full Stack internship for the period 01 June 2026 to 30 June 2026, an **Internship Letter** (July 2026) confirming successful completion of the internship, and a **Certificate of Attendance** (July 2026) confirming the project titled "Employee Management Analysis" carried out under the guidance of the organisation. The letters were issued by the Manager – Human Resources of Besant Technologies.

The training environment encouraged learning through explanation, demonstration, guided practice and project work. Working in such an environment helped in understanding how technical knowledge learnt in the classroom is applied in practical situations, and how discipline, punctuality and regular practice contribute to skill development.

## 1.5 Technologies Studied

The Java Full Stack internship covered the technologies required to build a complete web application, from the page that the user sees in the browser to the data that is stored in the database. The major technologies studied are described below.

**Java:** Java is a general-purpose, object-oriented programming language that is widely used for developing enterprise and web applications. Its important features include platform independence through the Java Virtual Machine (JVM), strong typing, automatic memory management, exception handling and a rich standard library. Concepts such as classes, objects, inheritance, polymorphism, encapsulation, abstraction, collections and exception handling form the foundation of back-end development.

**HTML:** HyperText Markup Language defines the structure of a web page. Elements such as headings, paragraphs, tables, forms, input fields and buttons are used to build the pages through which users enter and view employee information.

**CSS:** Cascading Style Sheets control the presentation of a web page, including colours, fonts, spacing, alignment and layout. CSS makes the application look consistent and helps the interface adapt to different screen sizes.

**JavaScript:** JavaScript adds behaviour to web pages. It is used for client-side form validation, handling button clicks, showing messages and updating parts of a page

without reloading it, which improves the user experience.

**SQL and Relational Databases:** Structured Query Language (SQL) is the standard language used to create, read, update and delete data in a relational database. Tables, primary keys, foreign keys, constraints, joins and aggregate functions are used to store employee information in an organised and consistent manner.

## 1.6 Scope of the Internship

The scope of the internship was to understand the complete development cycle of a small web application and to apply it to the project titled **Employee Management Analysis**. The areas covered during the internship include the following.

- Understanding the basics of Java programming and object-oriented concepts.
- Designing web pages using HTML and CSS and adding interactivity using JavaScript.
- Understanding the request-response cycle between the browser and the server.
- Designing a relational database and writing SQL queries for CRUD operations.
- Applying input validation to keep the stored data accurate and consistent.
- Analysing the requirements, architecture and modules of an employee management system.
- Preparing test cases and verifying the behaviour of the application.
- Documenting the work in the form of this internship report.

## 1.7 Internship Schedule

The internship was carried out over four weeks, from 01 June 2026 to 30 June 2026. The work was divided into weekly goals covering learning, design, implementation and documentation. The activities planned for each week are given in the table below, and the overall schedule is shown in Figure 1.1.

| Week | Activities |
|---|---|
| Week 1 | Java programming fundamentals, object-oriented concepts, collections and exception handling. |

| Week | Activities |
|---|---|
| Week 2 | HTML, CSS and JavaScript; designing web forms and client-side validation. |
| Week 3 | SQL and relational database design; database connectivity and CRUD operations. |
| Week 4 | Development, testing and documentation of the Employee Management Analysis project. |

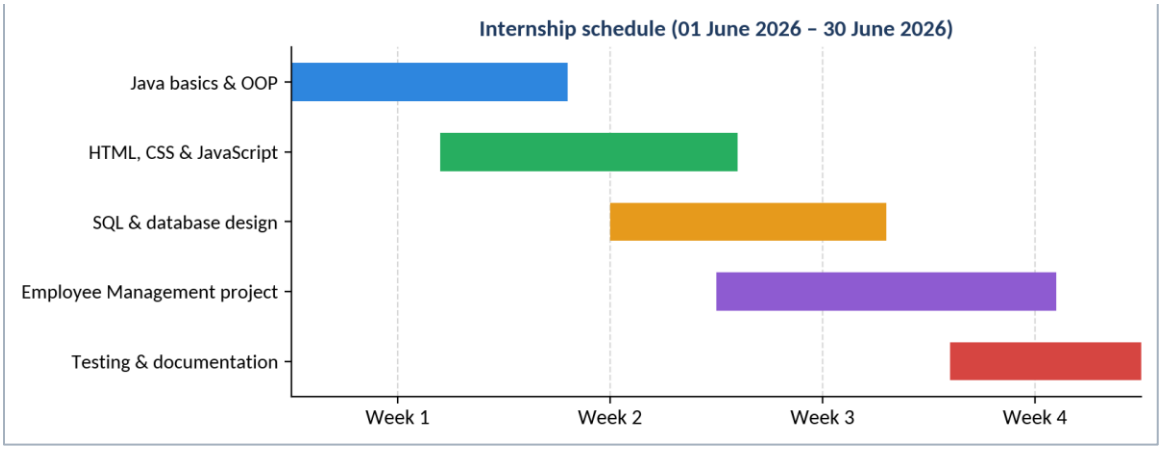



**Figure 1.1: Internship Schedule (June 2026)**

# CHAPTER 2: SYSTEM STUDY

## 2.1 Problem Statement Identification

Organizations manage large volumes of employee information that must be stored, updated, and retrieved efficiently. Manual record management across separate files often results in duplicate data, inconsistent records, time-consuming searches, and reduced productivity. As data increases, maintaining accuracy and generating reports become more difficult.

To overcome these challenges, organizations require a web-based employee management system that integrates **HTML**, **CSS**, and **JavaScript** for an interactive user interface with **Java** and **SQL** for application logic and secure data management. Such a system enables efficient storage, retrieval, updating, and deletion of employee records while maintaining data integrity.

The proposed solution improves data accessibility, reduces redundancy, supports faster report generation, and enhances overall organizational efficiency through effective database management.

## 2.2 Proposed Solution

The proposed solution is a web-based Employee Management Analysis system developed using the Java Full Stack approach. HTML, CSS, and JavaScript are used for creating an interactive and user-friendly interface, Java handles the application logic, and a relational database with SQL is used to store and manage employee information efficiently.

The system implements CRUD (Create, Read, Update, Delete) operations, allowing users to add, view, search, update, and delete employee records. Employee details such as Employee ID, Name, Department, Designation, Email Address, Joining Date, and Status are stored in the database with suitable constraints.

The development process includes requirement analysis, database design, interface design, implementation, testing, and result evaluation. The proposed system reduces manual effort, improves data accuracy, ensures efficient record management, and provides quick retrieval of employee information for better organizational productivity.

## 2.3 Existing System and Limitations

In many small and medium organisations, employee details are still maintained manually in registers or in separate spreadsheet files. Each department may keep its own copy of the data, and the information is updated by different people at different times. Although this method appears simple at the beginning, it becomes difficult to manage as the number of employees grows.

The major limitations of the existing manual system are listed below.

- **Data redundancy:** the same employee record is stored in more than one file.
- **Data inconsistency:** an update made in one file is not reflected in the other copies.
- **Slow retrieval:** searching for a particular employee requires opening many files or pages.
- **Human errors:** spelling mistakes, wrong dates and missing fields are common during manual entry.
- **Poor security:** files can be copied, shared or modified without any control.
- **Difficult reporting:** preparing department-wise or status-wise summaries takes considerable time.
- **No validation:** there is no automatic check to prevent duplicate or invalid entries.
- **Limited access control:** every person who has the file can view all the information.

## 2.4 Functional Requirements

The functional requirements describe what the Employee Management Analysis system must be able to do. They were identified by analysing the day-to-day activities involved in managing employee records.

| No. | Requirement | Description |
|---|---|---|
| FR1 | User Login | The system shall allow only registered users to log in using a username and password. |
| FR2 | Add Employee | The system shall allow an authorised user to add a new employee record with all the required details. |
| FR3 | View Employees | The system shall display the list of employees in a table with the important details. |
| FR4 | Search Employee | The system shall allow searching of employees by employee ID, name or department. |
| FR5 | Update Employee | The system shall allow modification of the details of an existing employee. |
| FR6 | Delete / Deactivate | The system shall allow removal or deactivation of an employee record after confirmation. |
| FR7 | Validation | The system shall check mandatory fields, data formats and duplicate employee IDs before saving. |
| FR8 | Reports | The system shall provide summary information such as the number of employees in each department. |

## 2.5 Non-Functional Requirements

Non-functional requirements describe the quality attributes that the system should possess in addition to its functions.

- **Usability:** the interface should be simple, with clear labels, and easy to use for a new user.
- **Performance:** common operations such as search and update should respond within a short time.
- **Reliability:** the system should behave predictably and should not lose data when an error occurs.
- **Security:** only authorised users should be able to access or change the employee data.
- **Maintainability:** the code should be organised into modules so that changes can be made easily.
- **Scalability:** the design should allow more records, users and features to be added in future.
- **Portability:** the application should run in common web browsers on different

operating systems.

## 2.6 Feasibility Study

A feasibility study was carried out to check whether the proposed system can be developed and used effectively.

**Technical Feasibility:** The technologies required for the project, namely Java, HTML, CSS, JavaScript and SQL, are open, well documented and supported by free development tools. The hardware required is an ordinary computer, so the system is technically feasible.

**Economic Feasibility:** The software tools used for development are available free of cost, and no special hardware is needed. The cost of development is therefore very low, while the savings in time and effort in maintaining employee records are significant.

**Operational Feasibility:** The interface is designed to be simple, so users who are familiar with basic computer usage can operate the system with minimum training. The system fits into the existing process of maintaining employee records and makes it faster and more accurate.

## 2.7 Hardware and Software Requirements

| Component | Requirement |
|---|---|
| Processor | Intel Core i3 or higher (or equivalent) |
| RAM | 4 GB minimum (8 GB recommended) |
| Hard Disk | Minimum 5 GB of free space |
| Operating System | Windows 10 / 11, Linux or macOS |
| Programming Language | Java (JDK 8 or higher) |
| Front-End Technologies | HTML, CSS and JavaScript |
| Database | Relational database with SQL support |
| Development Tools | Java IDE and a code editor |
| Web Browser | Google Chrome, Microsoft Edge or Mozilla Firefox |

## 2.8 Advantages of the Proposed System

- Employee information is stored in a single, central database.

- Duplicate and inconsistent records are avoided through validation and constraints.
- Searching and retrieving a record takes only a few seconds.
- Updates are reflected immediately for all users of the system.
- Access to the data is controlled through login and user roles.
- Summary reports can be prepared quickly and accurately.
- The modular design makes the system easy to maintain and extend.
- The use of open technologies keeps the cost of development low.

# CHAPTER 3: ARCHITECTURE

## 3.1 Architectural Design

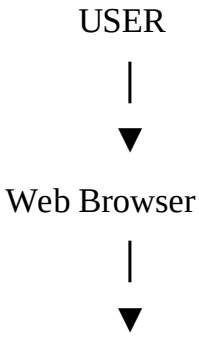
The **Employee Management Analysis** system follows a three-layer architecture consisting of the **Presentation Layer**, **Application Layer**, and **Database Layer**. The **Presentation Layer** is developed using **HTML, CSS, and JavaScript**, providing a user-friendly interface for managing employee records. Users can add, view, update, and delete employee information through the web application.

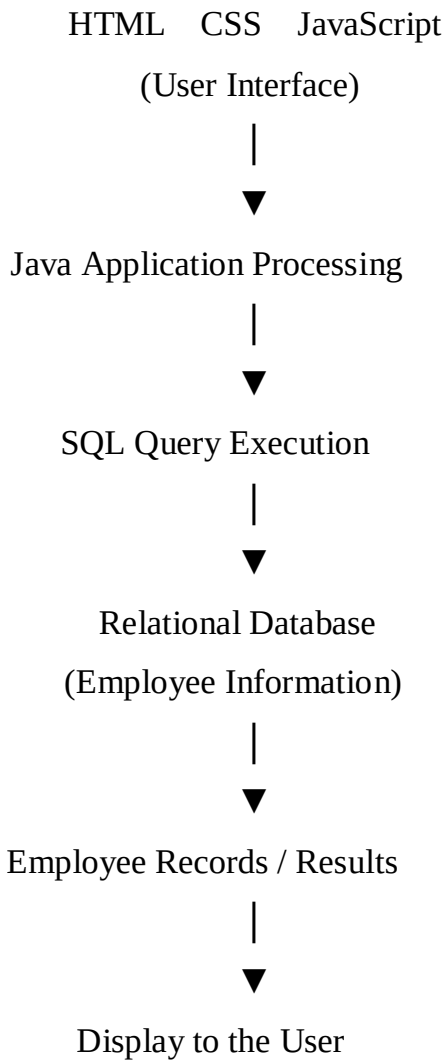
The **Application Layer**, developed in **Java**, processes user requests, validates input, and performs the required operations. The **Database Layer** stores employee information securely, while **SQL** queries are executed to manage and retrieve data efficiently.

**Working of the Architecture:**

1. The user accesses the application through a web browser.
2. Employee details are entered using the HTML, CSS, and JavaScript interface.
3. The Java application validates and processes the request.
4. SQL queries interact with the database to store or retrieve employee data.
5. The processed results are displayed to the user through the web interface.

**Architecture Flow Diagram**






The three-layer architecture described above is summarised in Figure 3.1. Each layer communicates only with the layer next to it: the browser sends HTTP requests to the Java application, and the Java application sends SQL queries to the database. The results travel

back in the opposite direction and are finally displayed to the user.

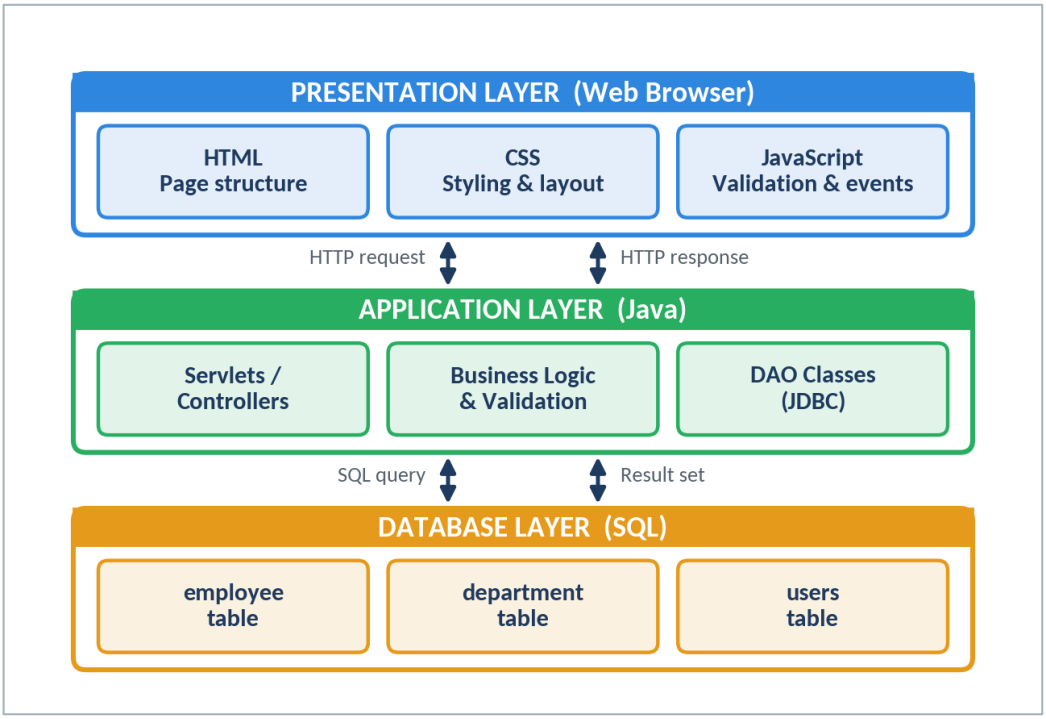



**Figure 3.1: Three-Layer Architecture of the System**

## 3.2 Modular Breakdown

The **Employee Management Analysis** system is divided into different functional modules. Each module performs a specific task, and together they ensure the smooth functioning of the application. The user interface is developed using **HTML, CSS, and JavaScript**, the logic using **Java**, while **SQL** is used for storing and managing employee information.

**Module 1: User Authentication Module**

This module allows users to securely access the application through login authentication. It verifies user credentials and ensures that only authorized users can use the system.

**Functions:**

- User Login
- User Authentication

- Secure Access to the System

**Module 2: Employee Management Module**

This module is responsible for managing employee information. Users can add new employee records and view or search existing details through web pages developed using **HTML, CSS, and JavaScript**.

**Functions:**

- Add Employee
- View Employee Details
- Search Employee Records

**Module 3: Employee Update Module**

This module allows users to modify existing employee information whenever required. Updated information is saved in the database using **SQL** operations.

**Functions:**

- Update Employee Details
- Edit Employee Information
- Save Updated Records

**Module 4: Employee Deletion Module**

This module enables users to remove employee records that are no longer required in the system while maintaining database consistency.

**Functions:**

- Delete Employee Records
- Remove Invalid or Duplicate Data
- Maintain Database Accuracy

**Module 5: Database Management Module**

This module manages employee information using a **relational database** and **SQL** for

performing CRUD (Create, Read, Update, Delete) operations. It ensures secure data storage, efficient retrieval, and accurate management of employee records.

**Functions:**

- Database Management using SQL
- CRUD Operations
- SQL Query Execution
- Data Storage and Retrieval

The modular design improves the organization, maintainability, and scalability of the **Employee Management Analysis** system. By integrating **Java, HTML, CSS, JavaScript, and SQL**, each module works together to provide an efficient, secure, and user-friendly application for managing employee information.

## 3.3 Database Design

The database is the foundation of the Employee Management Analysis system because every operation of the application reads or writes data. The database is designed using the relational model. The data is divided into tables, and the tables are linked to each other using keys so that the same information is not repeated in several places.

Three main entities are identified for the system: **Employee**, **Department** and **User**. The Employee table stores the personal and job-related details of each employee. The Department table stores the list of departments, and each employee belongs to exactly one department. The User table stores the login details of the people who are allowed to use the application.

| Entity | Key Attribute | Relationship |
|---|---|---|
| Employee | employee_id (Primary Key) | Many employees belong to one department (many-to-one). |
| Department | department_id (Primary Key) | One department has many employees (one-to-many). |
| User | user_id (Primary Key) | Each user has one role, which decides the permitted operations. |

The department_id column of the Employee table is a **foreign key** that refers to the Department table. This relationship ensures that an employee cannot be assigned to a department that does not exist, and that the department name is stored only once.

The relationship between the tables is shown in the entity-relationship diagram in Figure 3.2. The department_id attribute of the EMPLOYEE entity is a foreign key that refers to the primary key of the DEPARTMENT entity, so many employees can belong to one department. The USERS entity is independent of the other two and holds only the login information.

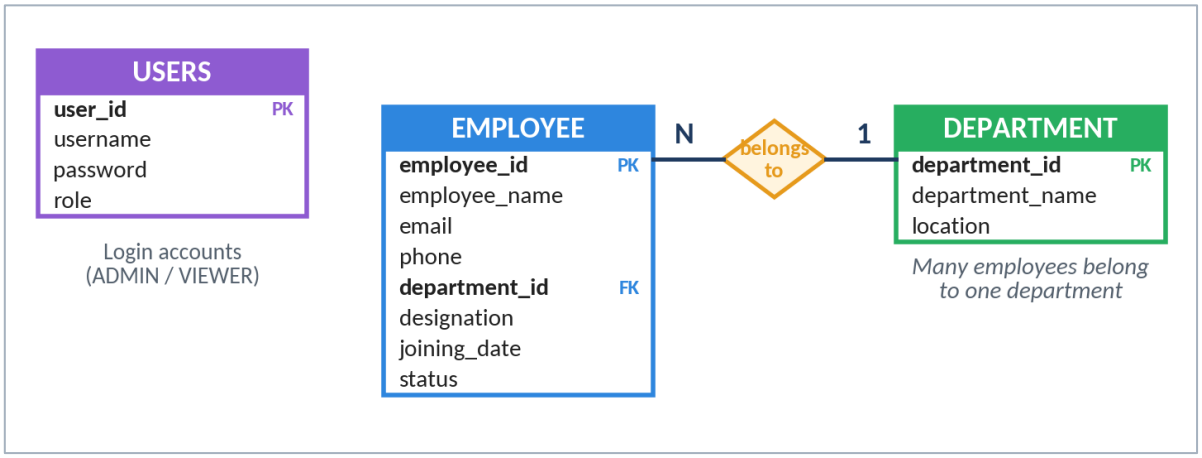



**Figure 3.2: Entity-Relationship Diagram**

## 3.4 Data Dictionary

The data dictionary describes each field of the tables, its data type and the constraint applied to it. The Employee table is shown first.

**Table: employee**

| Field | Data Type | Constraint | Description |
|---|---|---|---|
| employee_id | INT | PRIMARY KEY | Unique number of the employee |
| employee_name | VARCHAR(50) | NOT NULL | Full name of the employee |
| email | VARCHAR(80) | UNIQUE, NOT NULL | Official email address |
| phone | VARCHAR(15) | NOT NULL | Contact number |
| department_id | INT | FOREIGN KEY | Department of the employee |
| designation | VARCHAR(50) | NOT NULL | Job title of the employee |
| joining_date | DATE | NOT NULL | Date of joining the organisation |

| Field | Data Type | Constraint | Description |
|---|---|---|---|
| status | VARCHAR(10) | DEFAULT 'ACTIVE' | ACTIVE or INACTIVE |

**Table: department**

| Field | Data Type | Constraint | Description |
|---|---|---|---|
| department_id | INT | PRIMARY KEY | Unique number of the department |
| department_name | VARCHAR(50) | UNIQUE, NOT NULL | Name of the department |
| location | VARCHAR(50) | — | Place where the department works |

**Table: users**

| Field | Data Type | Constraint | Description |
|---|---|---|---|
| user_id | INT | PRIMARY KEY | Unique number of the user |
| username | VARCHAR(30) | UNIQUE, NOT NULL | Login name |
| password | VARCHAR(100) | NOT NULL | Password stored in encrypted form |
| role | VARCHAR(15) | NOT NULL | ADMIN or VIEWER |

## 3.5 Data Flow

The flow of data in the system can be understood at two levels. At the highest level, the user interacts with the Employee Management Analysis system, sends employee details or search requests, and receives results or messages in return.

At the next level, the following steps take place whenever a user performs an operation:

- The user enters data in a form or selects an action in the web page.
- JavaScript checks the entered values and informs the user if something is missing or wrongly formatted.
- The request is sent to the Java application, which validates the data again on the server side.
- The Java code prepares an SQL statement and sends it to the database.

- The database executes the statement and returns the result or an error message.
- The Java application converts the result into a response and sends it to the browser.
- The web page displays the result, for example the employee list or a success message.

The flow of data is shown diagrammatically in Figure 3.3 and Figure 3.4. In the context diagram (Level 0) the whole application is treated as a single process that exchanges data with the user and with the database.

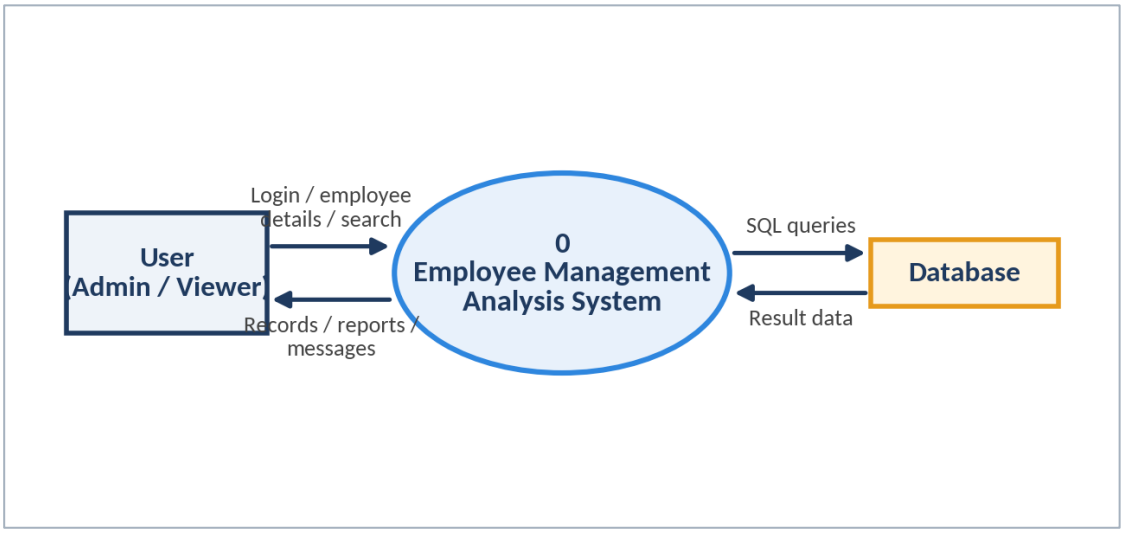



**Figure 3.3: Data Flow Diagram – Level 0 (Context Diagram)**

Level 1 splits the system into four processes. Authenticate User (1.0) reads the users data store, Manage Employees (2.0) reads and writes the employee data store, Search Employees (3.0) reads the employee data store, and Generate Reports (4.0) reads the

employee and department data stores to prepare the summary information.

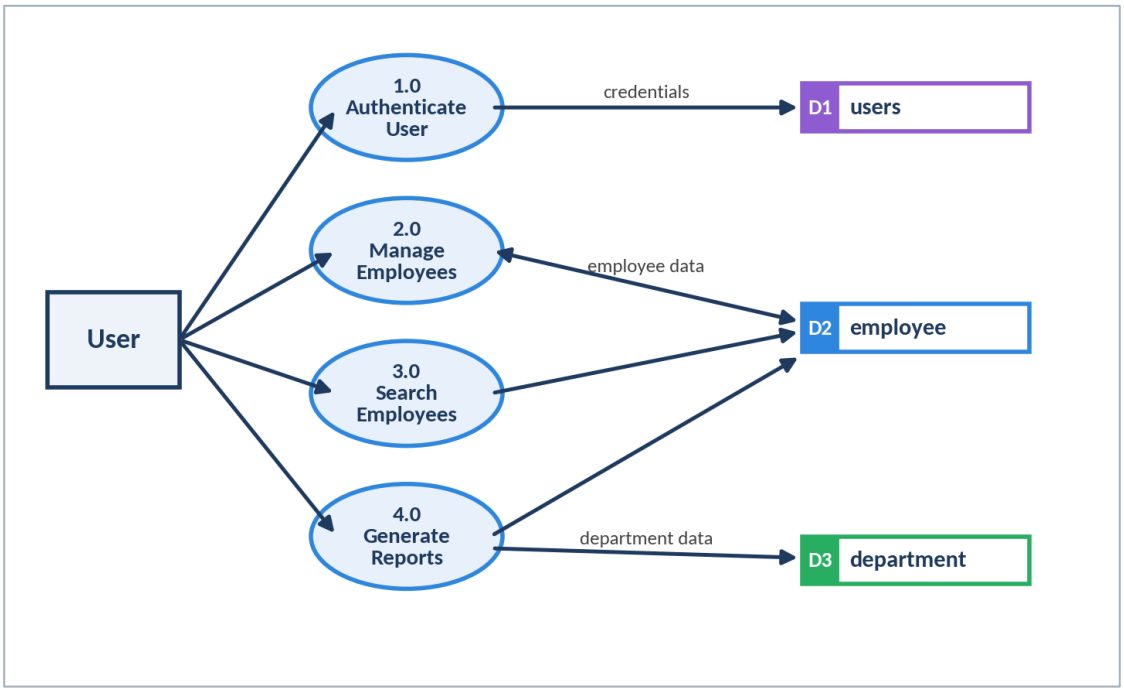



**Figure 3.4: Data Flow Diagram – Level 1**

## 3.6 User Roles and Access

Different users need different levels of access. The system therefore defines roles, and each role is allowed to perform only the operations that are required for it. Access is checked in the Java application and not only in the web page, because buttons that are hidden in the page can still be called directly.

| Operation | Admin | Viewer | Public | Remarks |
|---|---|---|---|---|
| Login | Yes | Yes | — | Valid credentials required |
| View employee list | Yes | Yes | No | After login only |
| Search employee | Yes | Yes | No | By ID, name or department |
| Add employee | Yes | No | No | Validated before saving |
| Update employee | Yes | No | No | Only permitted fields |
| Delete / deactivate | Yes | No | No | Confirmation required |

The permissions given in the table above are shown as a use-case diagram in Figure 3.5. The Admin can perform every operation, whereas the Viewer is limited to read-only operations such as logging in, viewing and searching employees, and viewing the

department report.

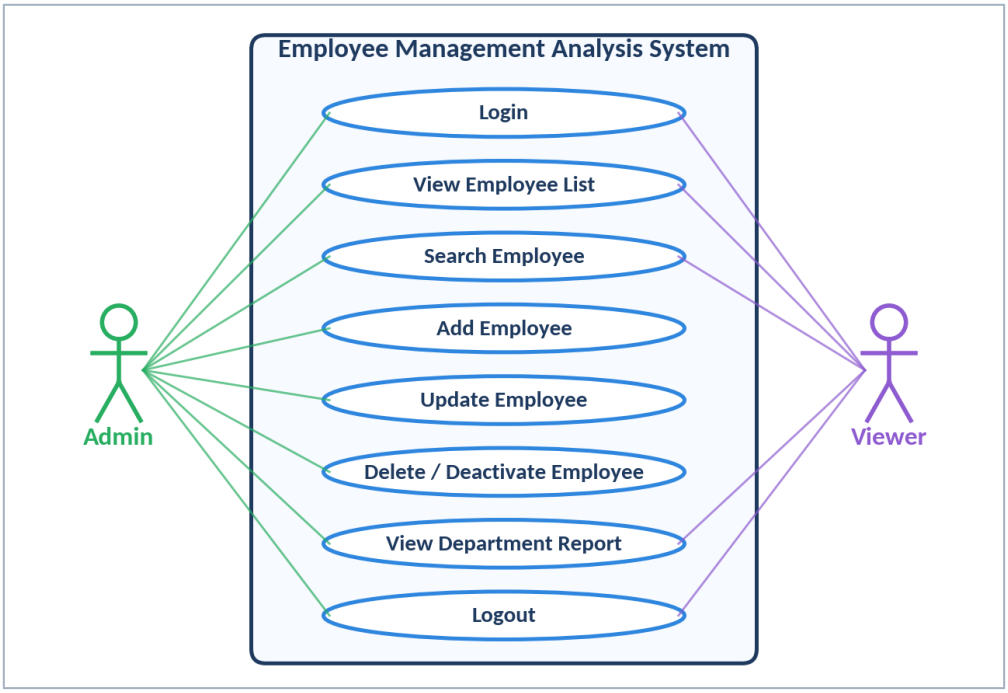



**Figure 3.5: Use-Case Diagram for Admin and Viewer**

## 3.7 Design Principles Followed

Several well-known design principles were followed while designing the system so that the application is easy to understand, test and extend.

- **Separation of concerns:** the user interface, the application logic and the data access are kept apart so that a change in one part has little effect on the others.
- **Model-View-Controller (MVC):** the model represents the data, the view displays it, and the controller handles the requests and connects the two.
- **Single responsibility:** each class and module performs one clearly defined task.
- **Validation at both ends:** data is checked in the browser for convenience and again on the server for safety.
- **Use of constraints:** primary keys, foreign keys and unique constraints in the database protect the data even if an error occurs in the application.
- **Least privilege:** each user is given only the access that is required for the work.

The way in which the MVC principle is applied in the system is shown in Figure 3.6. The controller receives each request from the browser, calls the model (the Employee class and the data-access class) to work with the database, and then selects the view that is sent back to the user.

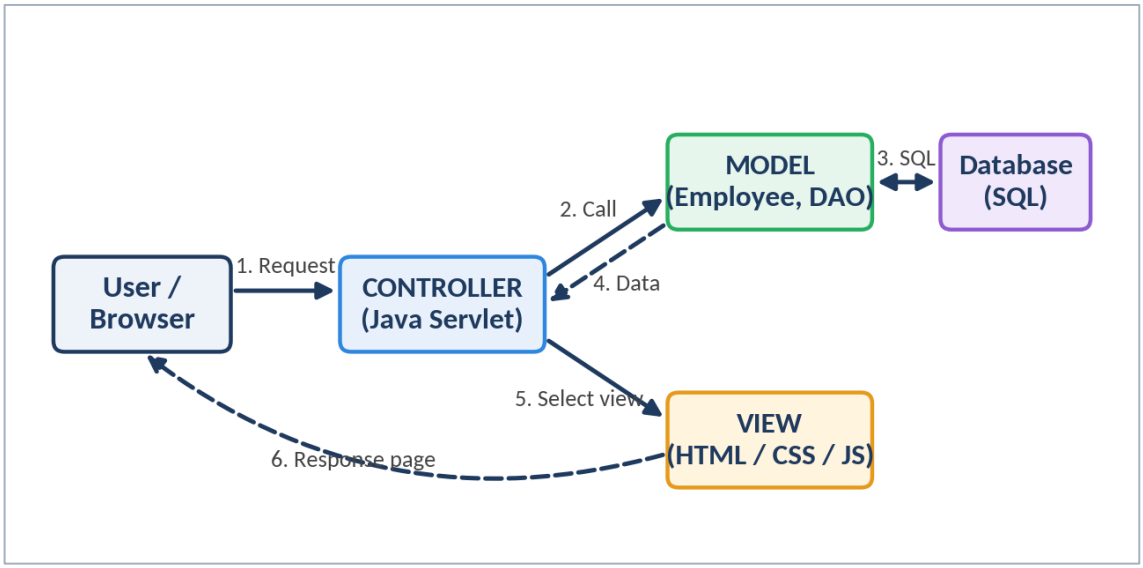



**Figure 3.6: Model-View-Controller Flow**

## 3.8 Page Navigation Flow

Figure 3.7 shows how a user moves between the pages of the application. After a successful login the user reaches the dashboard, from where the employee list, the add-employee form and the report page can be opened. Edit, delete and search operations are

started from the employee list, and the user can log out from the dashboard.

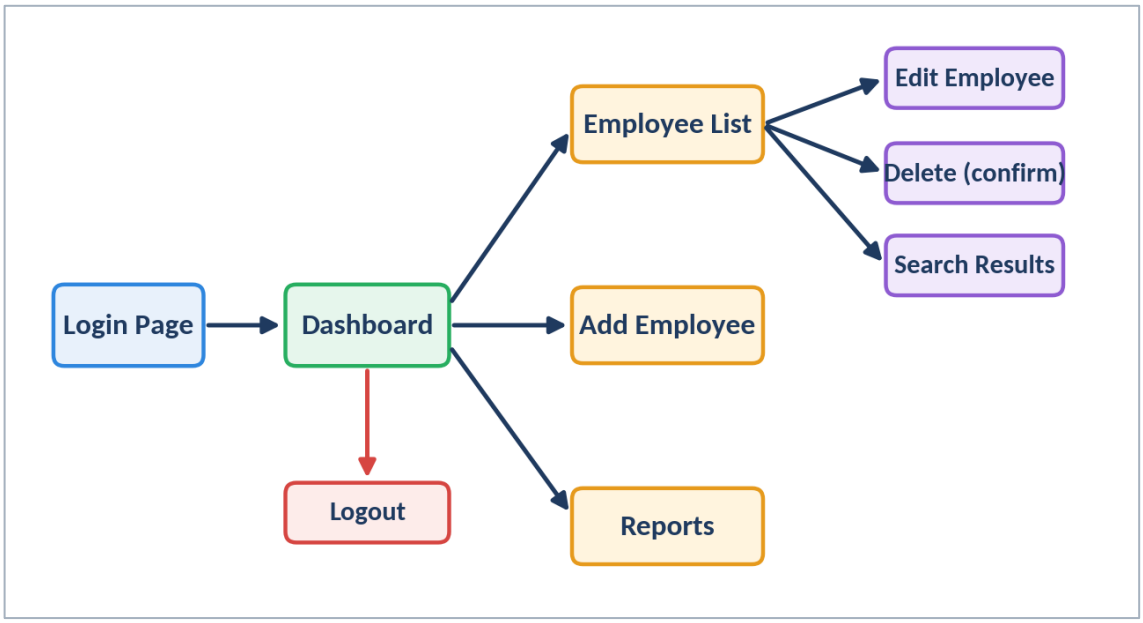



**Figure 3.7: Page Navigation Flow of the Application**

# CHAPTER 4: DEVELOPMENT AND RESULT ANALYSIS

## 4.1 System Implementation and Development

The implementation phase focused on developing a web-based **Employee Management Analysis** system using **Java, HTML, CSS, JavaScript, and SQL**. The system was designed to simplify employee record management through a responsive and user-friendly interface. Employee details such as Employee ID, Name, Department, Designation, Email, Joining Date, and Status were stored in the database with appropriate constraints to maintain data integrity.

The application supported **CRUD (Create, Read, Update, Delete)** operations using **SQL** queries. **HTML** and **CSS** were used to design the web pages, while **JavaScript** provided client-side validation and interactivity and **Java** handled server-side logic. The project was developed, tested, and verified to ensure accurate data storage, retrieval, updating, and deletion. Overall, the implementation provided practical experience in web development, database management, and application testing while enhancing technical and problem-solving skills.

## Development Environment

The project was developed on a personal computer using free tools. The tools used for each purpose are listed below.

| Purpose | Tool / Technology |
|---|---|
| Programming language | Java (JDK) |
| Code editing | Java IDE / code editor |
| Web pages | HTML5, CSS3, JavaScript |
| Database | Relational database server with SQL |
| Testing | Web browser and manual test cases |
| Documentation | Microsoft Word |

## Project Structure

The source code of the project is organised into separate packages for the model, the

data-access classes and the controllers, while the web pages, the style sheet and the JavaScript file are kept in the web application folder. This arrangement follows the separation of concerns principle described in Chapter 3. The structure of the project is shown in Figure 4.1.

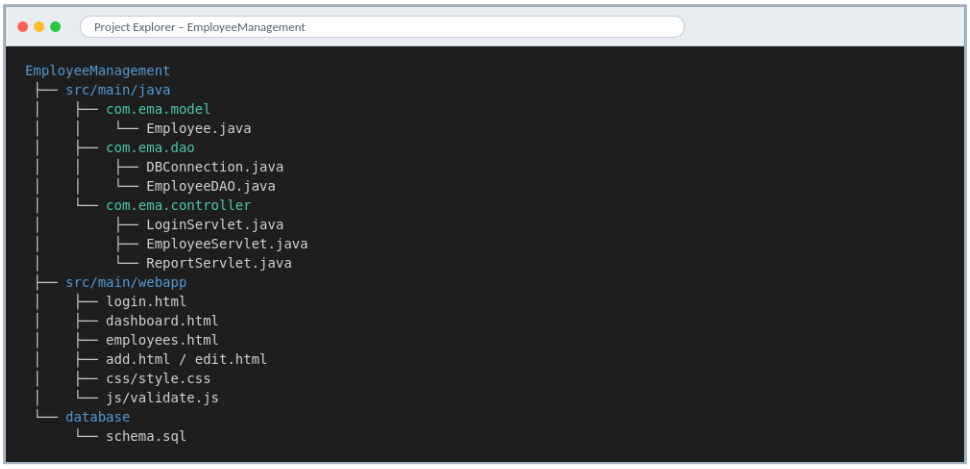



**Figure 4.1: Project Structure of the Application**

## Front-End Implementation

The front end consists of web pages for login, adding an employee, viewing the employee list and updating a record. Each page uses a simple and consistent layout. The employee form contains labelled input fields for the employee ID, name, email, phone number, department, designation and joining date. The structure of the form is shown below.

```
<form id="empForm" onsubmit="return validateForm()">
  <label>Employee ID</label>
  <input type="number" id="empId" required>
  <label>Name</label>
  <input type="text" id="empName" required>
  <label>Email</label>
  <input type="email" id="empEmail" required>
  <label>Department</label>
  <select id="dept">
    <option>Development</option>
    <option>Testing</option>
    <option>Support</option>
  </select>
  <button type="submit">Save Employee</button>
</form>
```

JavaScript is used to validate the form before it is submitted. The function checks that

the name is not empty and that the email address has a correct format. If a value is wrong, a message is shown to the user and the form is not submitted.

```
function validateForm() {
  var name  = document.getElementById("empName").value.trim();
  var email = document.getElementById("empEmail").value.trim();
  var pattern = /^[^@\s]+@[^@\s]+\.[^@\s]+$/;

  if (name === "") {
    alert("Employee name is required");
    return false;
  }
  if (!pattern.test(email)) {
    alert("Enter a valid email address");
    return false;
  }
  return true;
}
```

## Back-End Implementation

The back end is written in Java. A model class represents an employee, and a data-access class contains the methods that communicate with the database. Using a separate data-access class keeps the SQL statements away from the user-interface code. The model class is shown below.

```
public class Employee {
    private int id;
    private String name;
    private String email;
    private String department;

    public Employee(int id, String name,
                    String email, String department) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.department = department;
    }
    public int getId()            { return id; }
    public String getName()       { return name; }
    public String getEmail()      { return email; }
    public String getDepartment() { return department; }
}
```

The data-access method that inserts a new employee uses a **PreparedStatement**. A prepared statement treats the values as data and not as part of the SQL command, which protects the application from SQL injection.

```
public boolean addEmployee(Employee e) throws SQLException {
    String sql = "INSERT INTO employee "
               + "(employee_id, employee_name, email, department) "
               + "VALUES (?, ?, ?, ?)";
    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, e.getId());
        ps.setString(2, e.getName());
        ps.setString(3, e.getEmail());
        ps.setString(4, e.getDepartment());
        return ps.executeUpdate() > 0;
    }
}
```

Similar methods are written for retrieving, searching, updating and deleting employee records. Exceptions raised by the database, such as an attempt to insert a duplicate employee ID, are caught and converted into a clear message for the user instead of showing technical details.

## SQL Queries

[ADD YOUR OWN SQL QUERIES HERE – the queries you actually practiced during the internship. Delete this line after adding.]

### Sample SQL Statements for the Employee Table

The following statements show how the main CRUD operations are performed on the employee table described in the data dictionary.

```
CREATE TABLE employee (
  employee_id   INT PRIMARY KEY,
  employee_name VARCHAR(50) NOT NULL,
  email         VARCHAR(80) UNIQUE NOT NULL,
  department_id INT,
  designation   VARCHAR(50),
  joining_date  DATE,
  status        VARCHAR(10) DEFAULT 'ACTIVE'
);

-- Create
INSERT INTO employee VALUES
 (1, 'Arun', 'arun@example.com', 1, 'Developer', '2024-01-10', 'ACTIVE');

-- Read
SELECT * FROM employee;
```

```
SELECT * FROM employee WHERE department_id = 1;

-- Update
UPDATE employee SET designation = 'Senior Developer'
WHERE employee_id = 1;

-- Delete / Deactivate
UPDATE employee SET status = 'INACTIVE' WHERE employee_id = 1;
DELETE FROM employee WHERE employee_id = 1;

-- Reports
SELECT department_id, COUNT(*) AS total_employees
FROM employee GROUP BY department_id;

SELECT e.employee_name, d.department_name
FROM employee e
INNER JOIN department d ON e.department_id = d.department_id;
```

**SQL Operations Performed**

| SQL Concept | Purpose |
|---|---|
| CREATE DATABASE | Create a new database |
| CREATE TABLE | Create employee table |
| INSERT | Add employee records |
| SELECT | Retrieve employee information |
| UPDATE | Modify employee records |
| DELETE | Remove records |
| WHERE | Filter records |
| ORDER BY | Sort data |
| GROUP BY | Group records |

| | |
|---|---|
| HAVING | Filter grouped records |
| Aggregate Functions | Perform calculations |
| JOINS | Combine multiple tables |
| Subqueries | Execute nested queries |
| Views | Create virtual tables |

## 4.2 Result Analysis and Testing Parameters

The **Employee Management Analysis** system was tested to verify its functionality, accuracy, and performance. The application performed **CRUD (Create, Read, Update, Delete)** operations, ensuring efficient management of employee records. The user interface developed using **HTML, CSS, and JavaScript** was tested for responsiveness, smooth navigation, and proper form validation.

All **SQL** operations, including data insertion, retrieval, updating, and deletion, were checked for correct results. Employee records were stored accurately while maintaining data consistency and integrity, and invalid or duplicate entries were handled properly.

## Result Analysis Summary

The results of the project were analysed against the requirements listed in Chapter 2. The table below summarises each requirement and the way in which it was checked.

| Requirement | How it was checked | Expected outcome |
|---|---|---|
| Data storage | Adding records through the form and viewing them in the list | Every saved record appears in the list |

| Requirement | How it was checked | Expected outcome |
|---|---|---|
| Data retrieval | Searching with ID, name and department | Only matching records are displayed |
| Data update | Editing a record and reloading the page | The new value is retained |
| Data deletion | Deleting or deactivating a record | The record no longer appears as active |
| Validation | Submitting empty, duplicate or wrongly formatted values | A clear error message is shown |
| Access control | Trying restricted operations as a viewer | The operation is not permitted |

The analysis shows that the modular design, the use of validation at both the client and the server, and the use of database constraints together help to keep the employee data accurate and consistent.

**Testing Parameters**

- Verification of user login and authentication.
- Validation of employee registration forms.
- Testing of CRUD operations.
- Verification of SQL query execution.
- Testing of database operations.
- Validation of data retrieval and record updates.
- User interface testing using HTML, CSS, and JavaScript.

## 4.3 Test Cases

Test cases were prepared to check the important functions of the system with valid and invalid inputs. Each test case states the input, the expected result and the area that is being verified.

| ID | Test Case | Input | Expected Result |
|---|---|---|---|
| TC-01 | Valid login | Correct username and password | User is taken to the home page |
| TC-02 | Invalid login | Wrong password | Error message is displayed |

| ID | Test Case | Input | Expected Result |
|---|---|---|---|
| TC-03 | Add employee | All fields filled correctly | Record is saved and appears in the list |
| TC-04 | Empty name | Name field left blank | Form is rejected with a message |
| TC-05 | Duplicate ID | An employee ID that already exists | Duplicate is rejected |
| TC-06 | Invalid email | Email without the @ symbol | Validation message is displayed |
| TC-07 | Search employee | An existing employee ID | Correct record is displayed |
| TC-08 | Search unknown | An ID that does not exist | "No records found" message |
| TC-09 | Update employee | Change of designation | New designation is saved |
| TC-10 | Delete employee | Delete with confirmation | Record is removed from the list |
| TC-11 | Special characters | Quotes and symbols in the name | Value is stored and shown safely |

## 4.4 Challenges and Solutions

Several practical difficulties were faced while learning and working on the project. Overcoming them improved the understanding of the subject.

| Challenge | Solution |
|---|---|
| Understanding how the browser, Java code and database are connected | Studied the request-response cycle and traced one operation from the form to the database step by step. |
| Errors in SQL statements | Tested each query separately on sample data before using it in the program. |
| Duplicate and incomplete data | Added mandatory fields, unique constraints and validation messages. |
| Mixing logic and SQL in one place | Moved the database code into a separate data-access class. |
| Time management in a one-month period | Divided the work into weekly goals for learning, design, implementation and documentation. |

## 4.5 Output Screens

This section presents the output screens of the Employee Management Analysis system. The screens are shown in the order in which a user normally works with the application, starting from the login page and ending with the report and the access control screens. The output of the SQL statements executed on the database is given in Section 4.6.

### 4.5.1 Login Page

The login page is the entry point of the application. The user enters a username and a password, and the Login button sends them to the Java application, which checks them against the users table. The password is masked while it is typed, and only registered users are allowed to proceed to the dashboard.

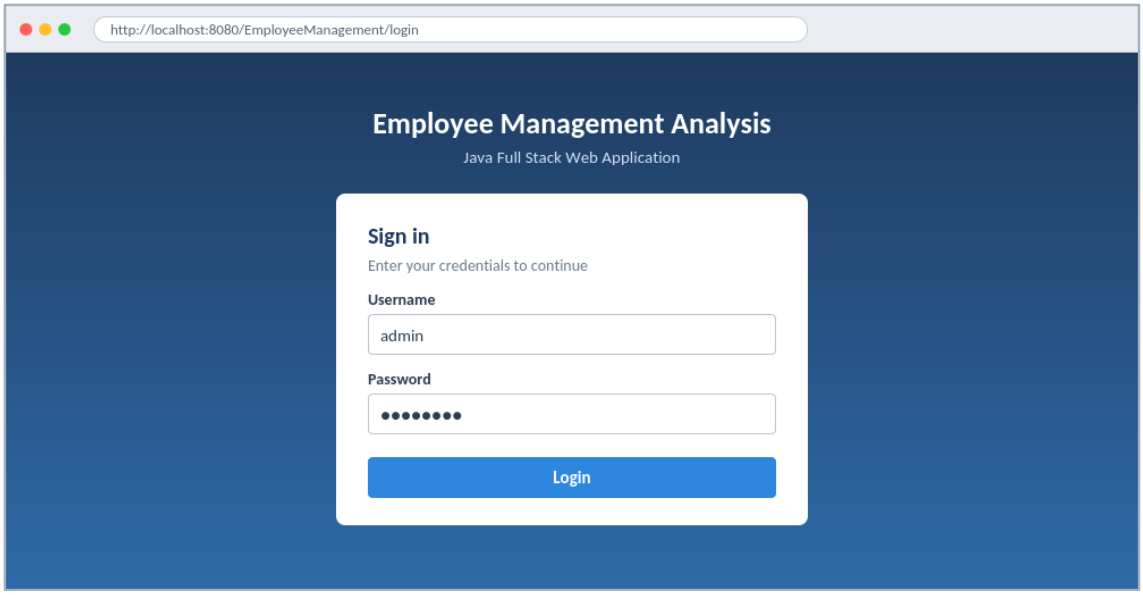



**Figure 4.2: Login Page**

### 4.5.2 Login Validation

When the username or password is wrong, the user is not allowed to enter the system. The page stays on the login screen, the password field is highlighted and a clear message is displayed. This screen corresponds to the invalid login test case (TC-02), and no

technical details of the failure are shown to the user.

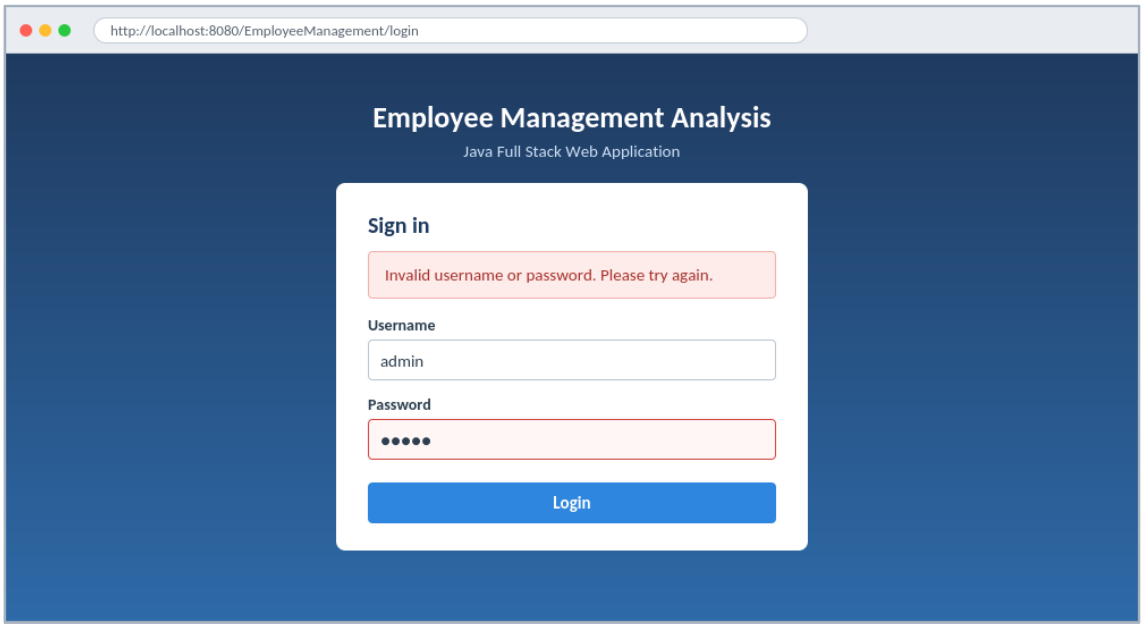



**Figure 4.3: Error Message for Invalid Login**

### 4.5.3 Dashboard

After a successful login the user is taken to the dashboard. It shows the total number of employees, the number of active and inactive employees and the number of departments. A bar chart shows the number of employees in each department, and the

quick action buttons give direct access to the most common operations.

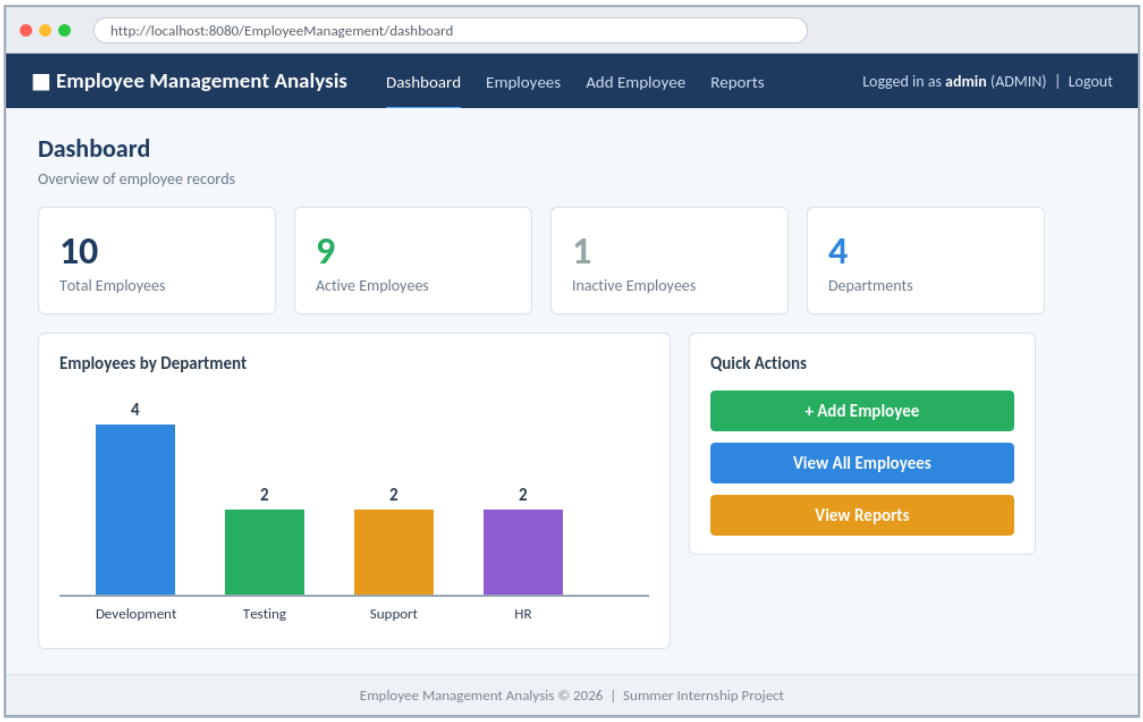



**Figure 4.4: Dashboard of the Application**

### 4.5.4 Employee List

The employee list displays all the stored records in a table with the employee ID, name, email, department, designation, joining date and status. The status is shown as a coloured label so that active and inactive employees can be identified at a glance. The search box, the department filter, the Add Employee button and the Edit and Delete

buttons for each record are available on this page.

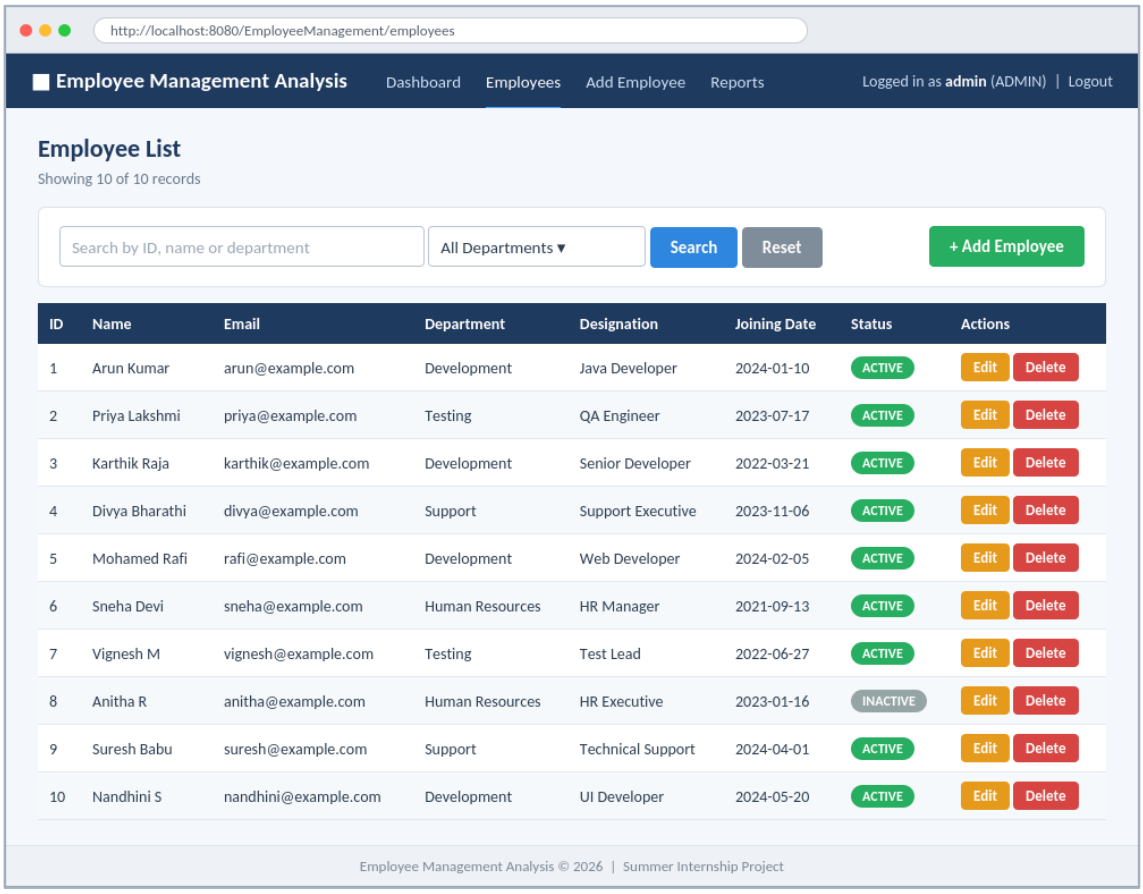



**Figure 4.5: Employee List Page**

### 4.5.5 Add Employee Form

The add employee form is used to enter the details of a new employee. The fields marked with an asterisk are mandatory. The department is selected from a drop-down list, and the joining date is entered in the YYYY-MM-DD format. The Save Employee button

submits the form, and the Cancel button returns to the list without saving.

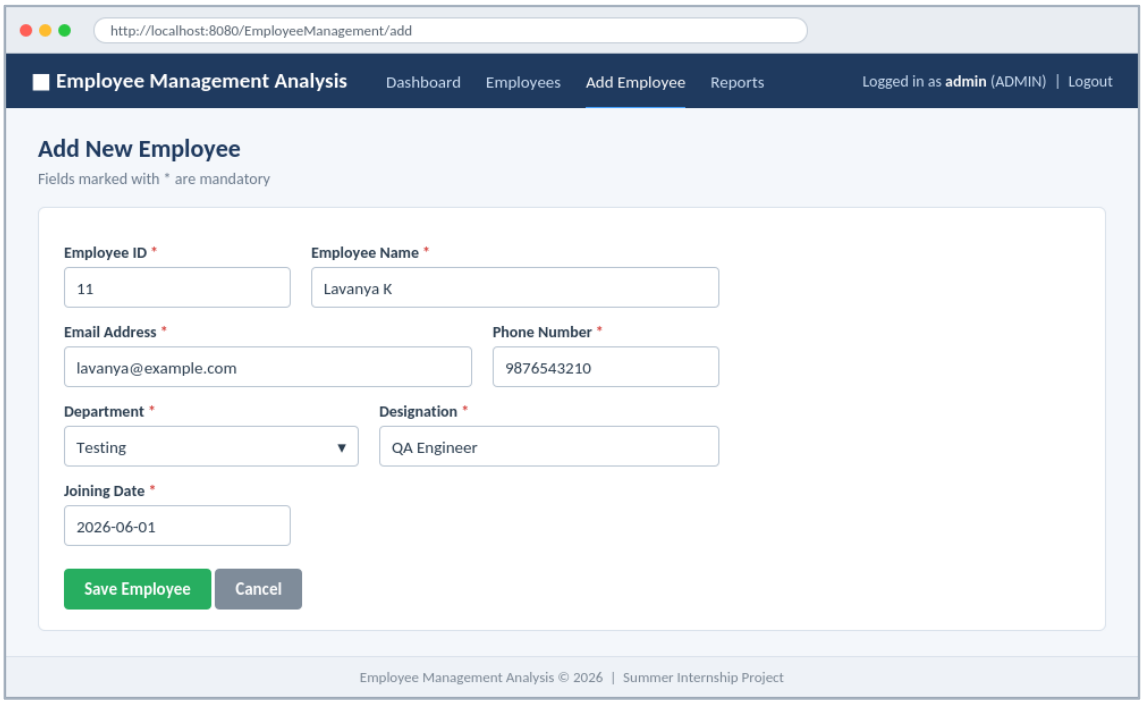



**Figure 4.6: Add Employee Form**

### 4.5.6 Form Validation

JavaScript checks the form before it is submitted. In this screen the employee name is left blank and the email address does not contain the @ symbol, so the two fields are highlighted and a message is shown below each of them. The form is not submitted until

the errors are corrected. This screen corresponds to the test cases TC-04 and TC-06.

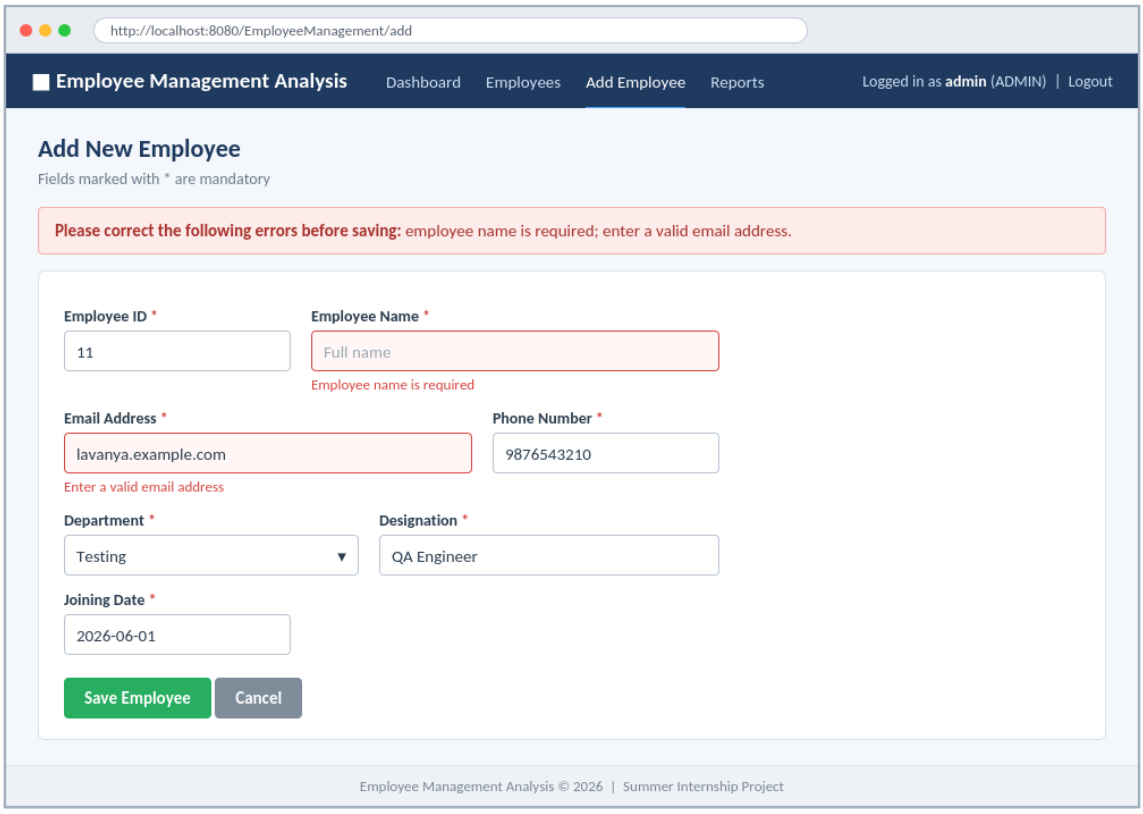



**Figure 4.7: Client-Side Validation Messages**

### 4.5.7 Duplicate Employee ID

Each employee ID must be unique. When an ID that already exists is entered, the server-side validation and the primary key constraint of the database prevent the second record from being stored, and a simple message is displayed instead of a technical error.

This screen corresponds to the test case TC-05.

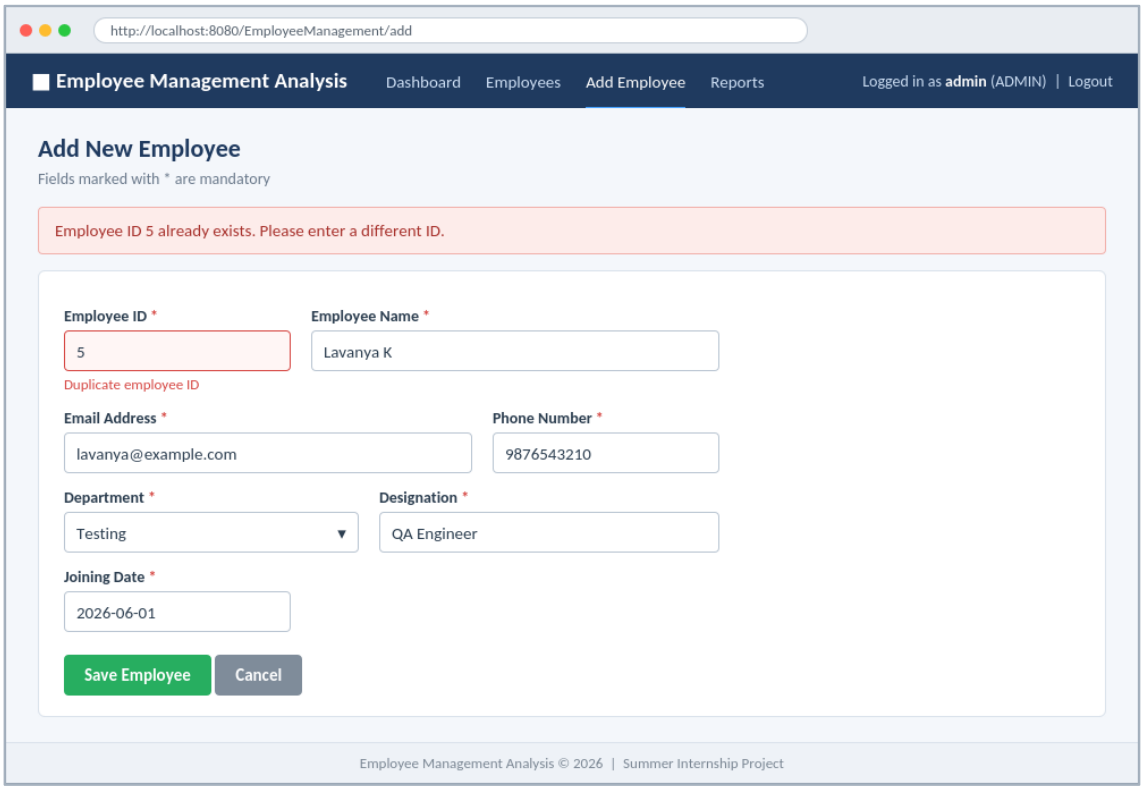



**Figure 4.8: Duplicate Employee ID Rejected**

### 4.5.8 Record Saved Successfully

When all the values are valid, the record is stored in the database and the user is returned to the employee list. A success message is displayed and the newly added employee appears in the list, highlighted so that it can be found easily. This screen

corresponds to the test case TC-03.

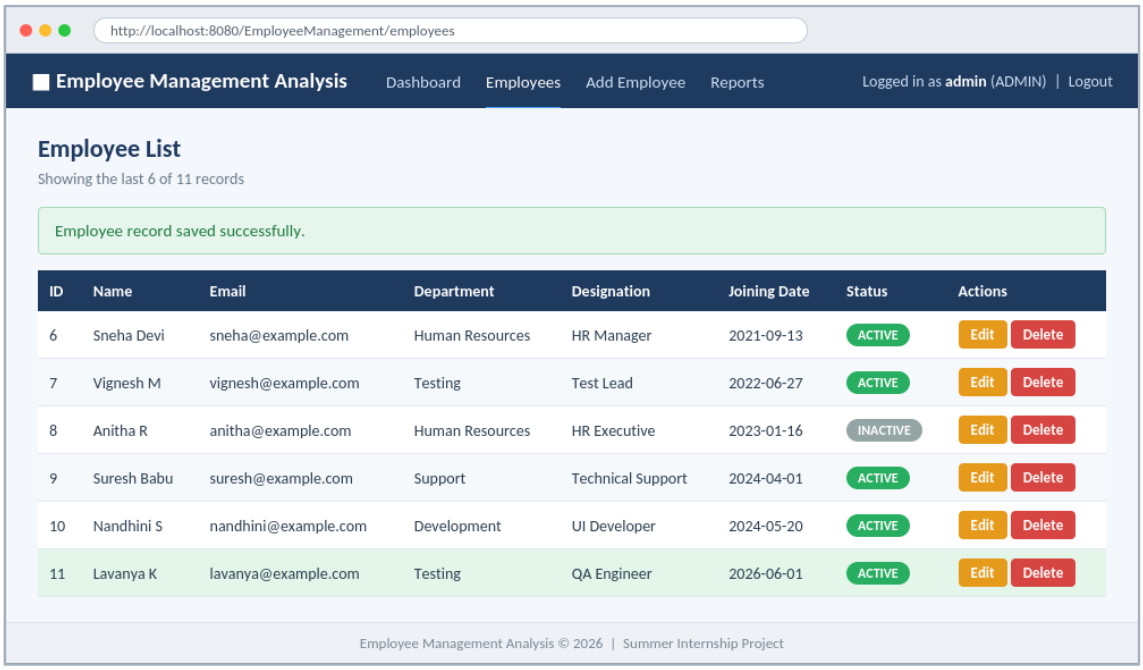



**Figure 4.9: Success Message after Adding an Employee**

### 4.5.9 Search by Department

Employees can be searched by employee ID, name or department. In this screen the Development department is selected, and only the four employees of that department are displayed along with the number of records found. This screen corresponds to the test

case TC-07.

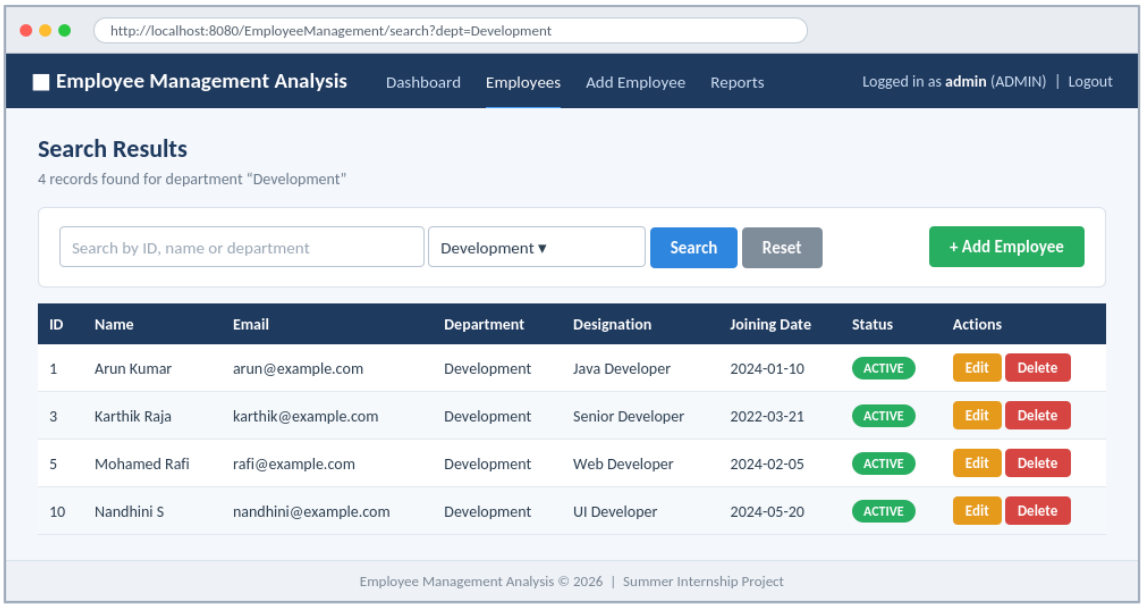



**Figure 4.10: Search Result for the Development Department**

### 4.5.10 Search with No Result

When the entered ID or name does not match any record, the system does not show an empty table. Instead, a clear No records found message is displayed so that the user knows that the search was completed. This screen corresponds to the test case TC-08.

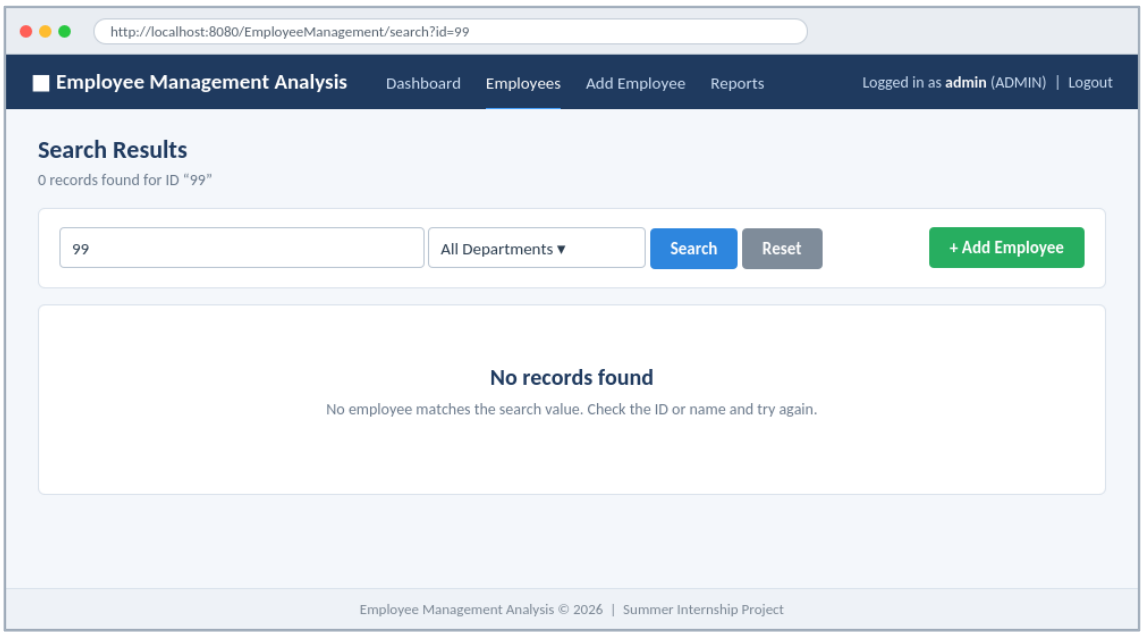



**Figure 4.11: No Records Found Message**

### 4.5.11 Edit Employee

The edit form opens with the existing details of the selected employee already filled in. The employee ID is shown as a read-only field because it is the primary key and must not be changed. In this example the designation is being changed from Java Developer to Senior Developer.

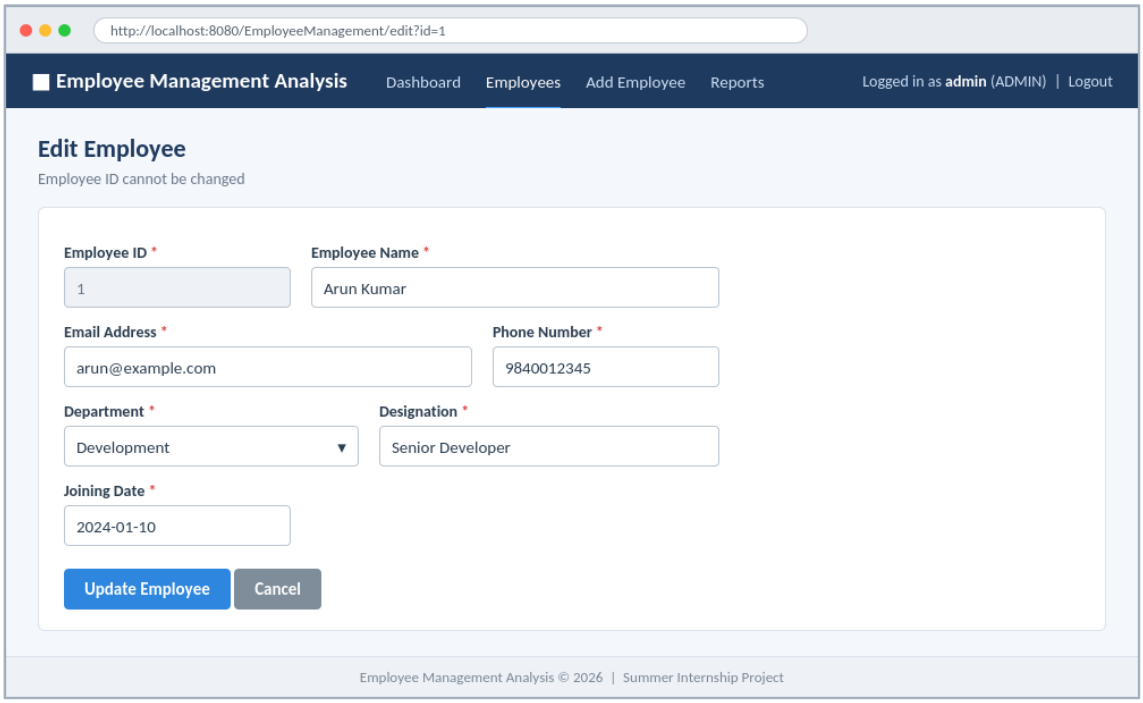



**Figure 4.12: Edit Employee Form**

### 4.5.12 Update Confirmation

After the Update Employee button is pressed, the changed values are validated and saved using an SQL UPDATE statement. The list is displayed again with a success message, and the updated record shows the new designation. This screen corresponds to

the test case TC-09.

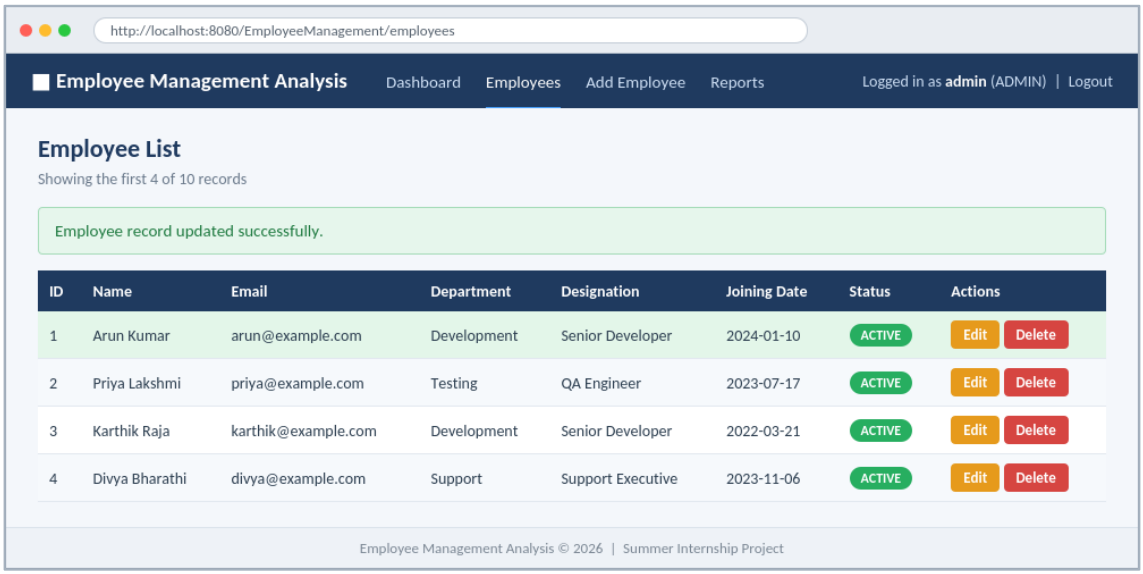



**Figure 4.13: Success Message after Updating an Employee**

### 4.5.13 Delete Confirmation

Deleting a record cannot be undone, so the system asks for confirmation before the operation is carried out. The dialog shows the ID, name, department and designation of the selected employee, and the record is removed only when the user chooses Yes, Delete. The Cancel button closes the dialog without any change. This screen corresponds

to the test case TC-10.

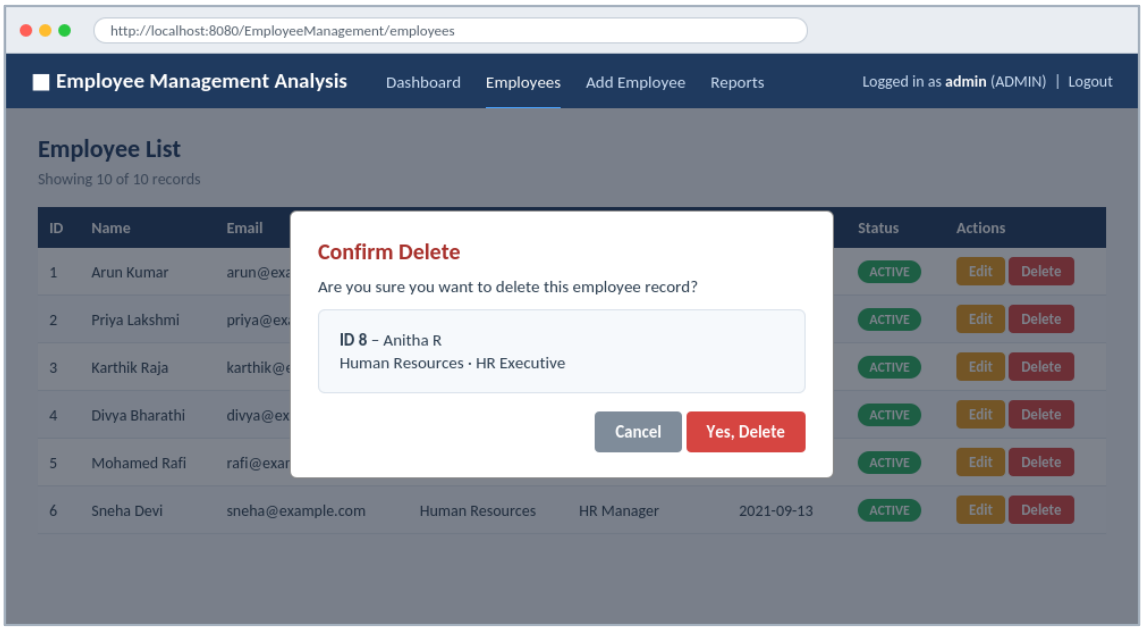



**Figure 4.14: Confirmation Dialog before Deleting a Record**

### 4.5.14 Department-wise Report

The report page gives a summary of the employee data. A table shows the total, active and inactive employees of each department together with the overall totals, and a bar chart shows the same distribution visually. The report is prepared with the SQL

GROUP BY and COUNT functions and meets the reporting requirement FR8.

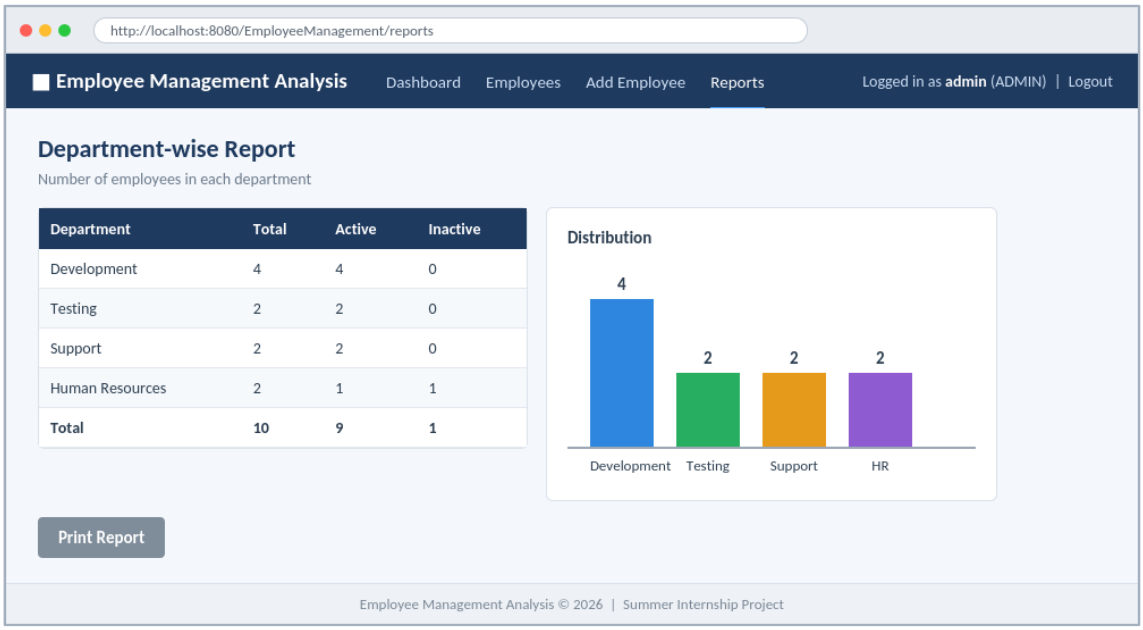



**Figure 4.15: Department-wise Employee Report**

### 4.5.15 Viewer Role Screen

A user with the Viewer role can log in, see the employee list and use the search facility, but the Add, Edit and Delete options are not shown. This is an application of the least privilege principle described in Section 3.7, and it ensures that viewers can read the

data but cannot change it.

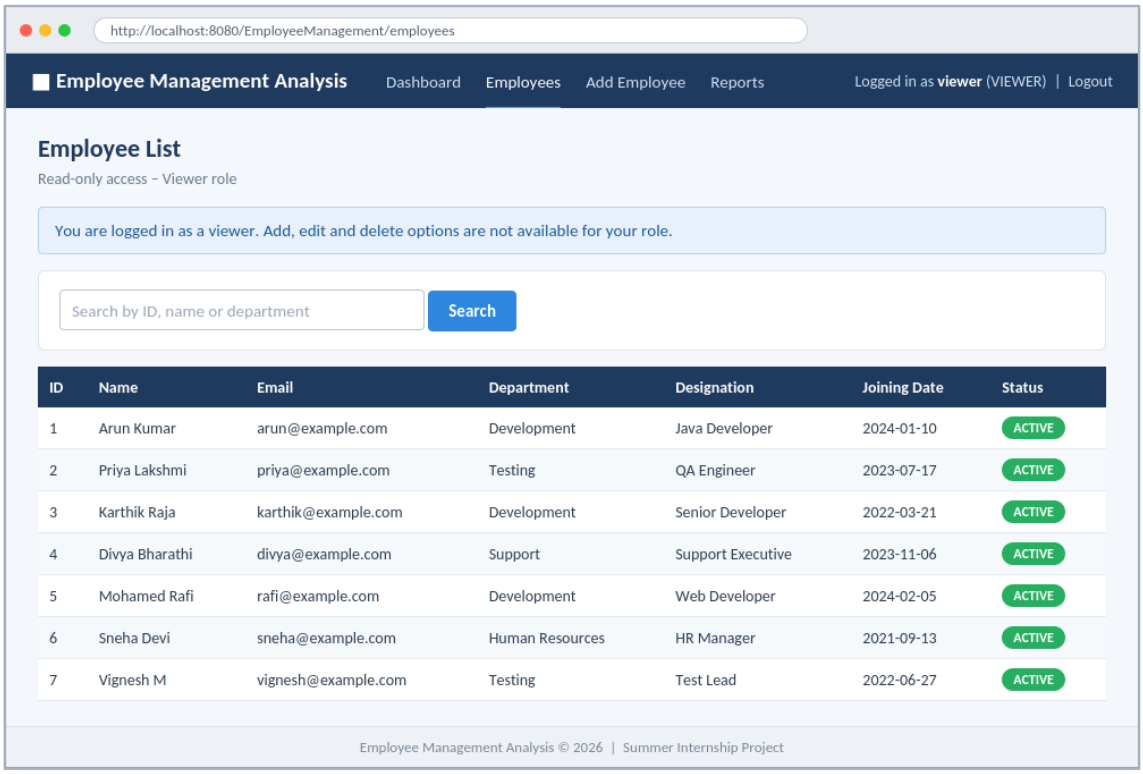



**Figure 4.16: Employee List for a Viewer**

### 4.5.16 Access Denied

Hiding the buttons is not sufficient for security, because a restricted page can still be requested directly by typing its address in the browser. The Java application therefore checks the role of the user again, and when a viewer tries to perform a restricted

operation an Access Denied page is displayed and no change is made to the data.

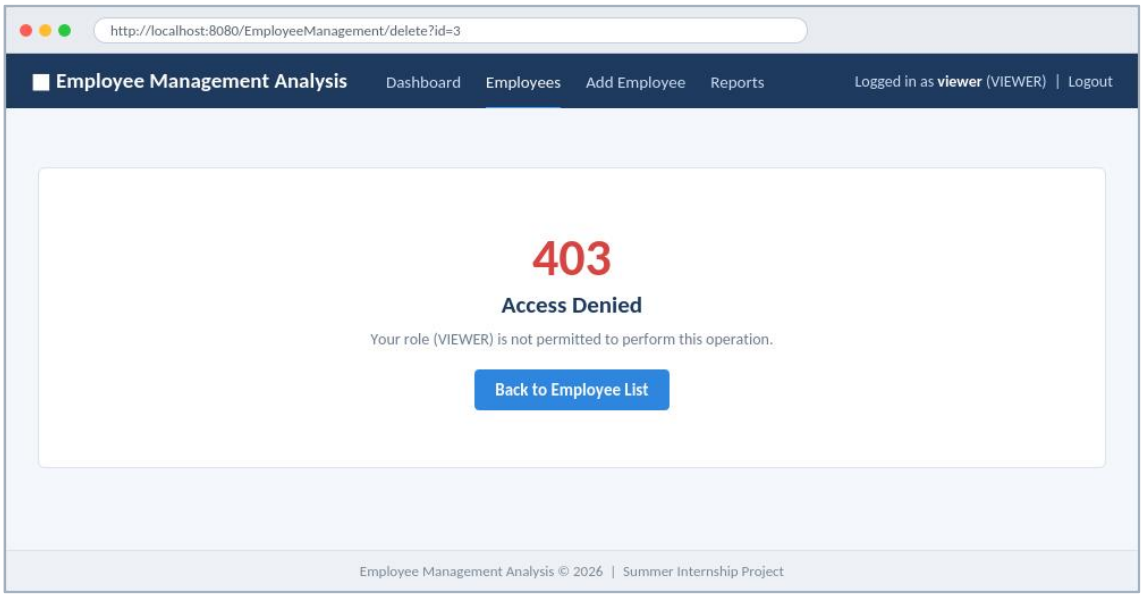



**Figure 4.17: Access Denied Message for a Restricted Operation**

## 4.6 Database Output

The following screens show the SQL statements that were executed on the employee table and the output returned by the database. They demonstrate table creation, insertion, retrieval, updating, grouping and joining of the data described in Section 3.4.

### 4.6.1 Creating the Table and Inserting Records

The employee table is created with the primary key, the unique constraint on the email and the default value for the status. The first INSERT statement adds a record successfully, whereas the second INSERT statement uses the same employee ID and is rejected by the database because of the primary key constraint.

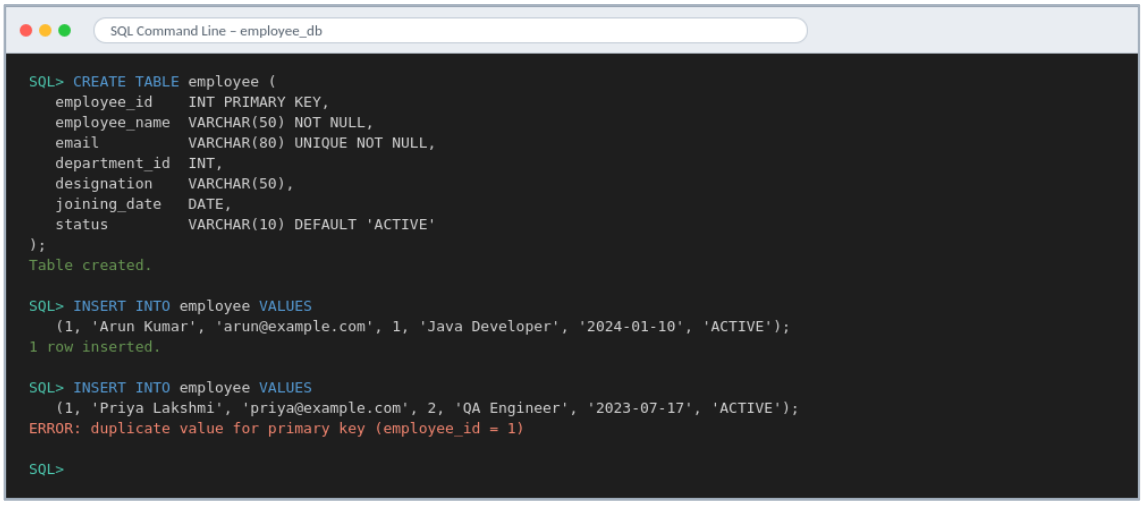



**Figure 4.18: Output of CREATE TABLE and INSERT Statements**

### 4.6.2 Retrieving and Updating Records

The SELECT statement with a WHERE condition retrieves only the employees of the department with the ID 1, and the number of rows selected is displayed below the table. The UPDATE statement then changes the designation of the employee with the ID 1, and

the database reports that one row was updated.

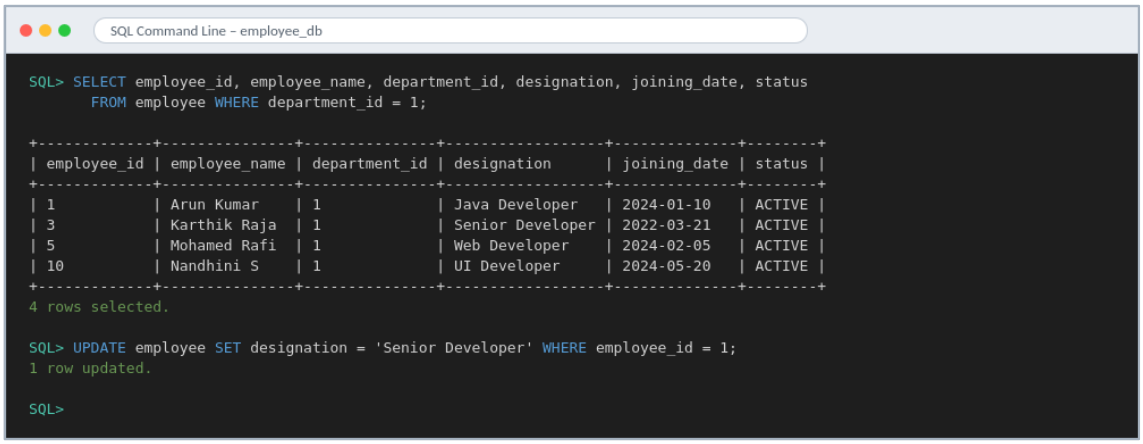



**Figure 4.19: Output of SELECT with WHERE and UPDATE**

### 4.6.3 Grouping Records

The GROUP BY clause together with the COUNT function gives the number of employees in each department. This is the query that supplies the values shown in the department-wise report of the application.

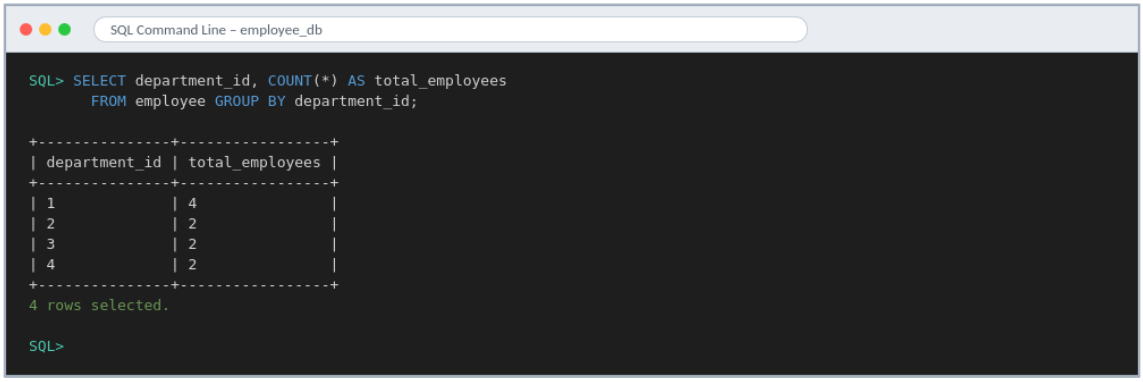



**Figure 4.20: Output of GROUP BY with COUNT**

### 4.6.4 Joining Tables

The INNER JOIN combines the employee table and the department table using the department_id column, so that the name of the department is displayed along with each

employee name instead of the department number.

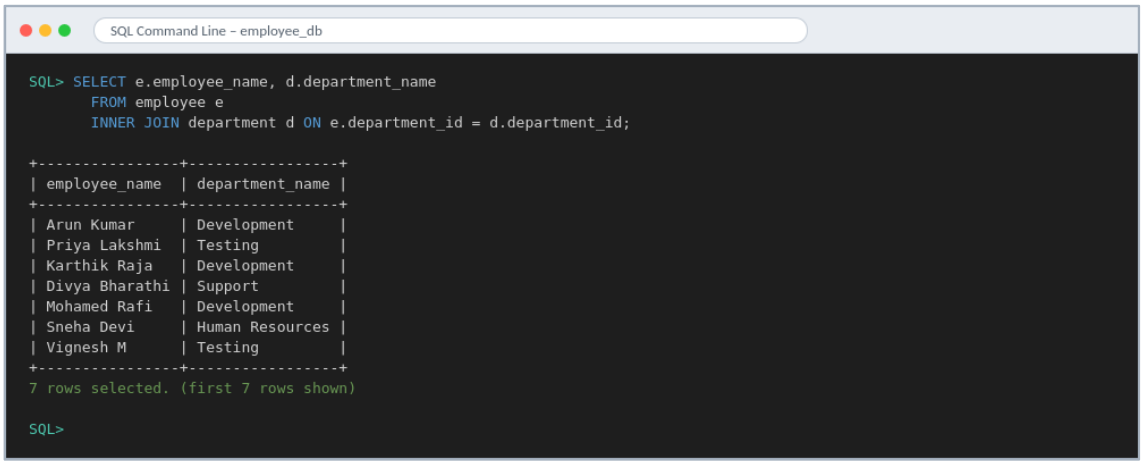



**Figure 4.21: Output of INNER JOIN between employee and department**

## 4.7 Summary of Output Screens

The table below relates each output screen to the requirement or the test case that it illustrates.

| Figure No. | Screen | Requirement / Test Case |
|---|---|---|
| 4.2, 4.3 | Login and invalid login | FR1; TC-01, TC-02 |
| 4.4 | Dashboard | FR3, FR8 |
| 4.5 | Employee list | FR3 |
| 4.6 | Add employee form | FR2 |
| 4.7, 4.8 | Validation and duplicate ID | FR7; TC-04, TC-05, TC-06 |
| 4.9 | Record saved successfully | FR2; TC-03 |
| 4.10, 4.11 | Search and no result | FR4; TC-07, TC-08 |
| 4.12, 4.13 | Edit and update | FR5; TC-09 |
| 4.14 | Delete confirmation | FR6; TC-10 |
| 4.15 | Department-wise report | FR8 |
| 4.16, 4.17 | Viewer role and access denied | Section 3.6 – User Roles and Access |
| 4.18 – 4.21 | SQL output screens | FR2 to FR8 (database operations) |

# CHAPTER 5: CONCLUSION

## 5.1 Conclusion

The internship at **Besant Technologies** provided valuable practical knowledge in **Java, HTML, CSS, JavaScript, and SQL**. It enhanced my understanding of web application development, user interface design, and database management through hands-on learning. The training helped improve my analytical thinking, logical reasoning, problem-solving abilities, and technical skills while providing exposure to real-world software development practices.

The project titled **"Employee Management Analysis"** demonstrated how CRUD operations can be used to manage employee records efficiently. **HTML, CSS, and JavaScript** were used to develop an interactive user interface, **Java** for the application logic, and **SQL** for data storage and management. Overall, the internship bridged the gap between academic learning and industry practices, establishing a strong foundation for future academic projects and professional careers in web application development and database management.

## 5.2 Skills Gained

The one-month internship helped in developing both technical and professional skills. The important skills gained are listed below.

**Technical skills**

- Writing object-oriented programs in Java.
- Designing web pages using HTML and CSS.
- Adding validation and interactivity using JavaScript.
- Designing a relational database and writing SQL queries.
- Understanding the connection between the front end, the back end and the database.
- Preparing test cases and checking the behaviour of an application.

**Professional skills**

- Analysing a problem and breaking it into smaller tasks.
- Planning work and completing it within the available time.
- Documenting the work clearly in the form of a report.
- Communicating technical ideas in a simple way.
- Working with discipline and responsibility in a professional environment.

## 5.3 Future Enhancements

The Employee Management Analysis system can be extended in several ways in the future to make it more useful for an organisation.

- Adding attendance and leave management for the employees.
- Adding payroll calculation and salary slip generation.
- Providing graphical dashboards with charts for department-wise and status-wise analysis.
- Exporting reports to PDF and Excel files.
- Using a framework such as Spring Boot to develop a REST API for the back end.
- Adding email notifications for important changes in employee records.
- Improving security through password hashing and role-based dashboards.
- Making the interface fully responsive for mobile phones and tablets.

With these improvements, the system can grow from a learning project into a complete employee information system that supports the daily needs of an organisation.

## REFERENCES


1. Oracle Corporation, **Java Documentation** - https://docs.oracle.com/en/java/
2. Oracle Corporation, **Java JDBC Tutorial** - https://docs.oracle.com/javase/tutorial/jdbc/
3. MDN Web Docs (HTML, CSS, and JavaScript Documentation). https://developer.mozilla.org/

4. Besant Technologies Official Website - https://www.besanttechnologies.com/
5. W3Schools – Java Tutorial - https://www.w3schools.com/java/

## APPENDICES

### Appendix A – Internship Details

**Student:** SURJITH S

**Register Number:** 810023205031

**Institution:** University College of Engineering, BIT Campus, Anna University, Tiruchirappalli

**Organisation:** Besant Technologies, Chennai

**Domain:** Java Full Stack

**Project Title:** Employee Management Analysis

**Duration:** 01 June 2026 to 30 June 2026

**Training Time:** 10:00 AM to 04:00 PM

### Appendix B – Certificates

[Attach scanned copies of Offer Letter, Internship Letter and Certificate of Attendance here.]